AI VIET NAM
@aivietnam.edu.vn

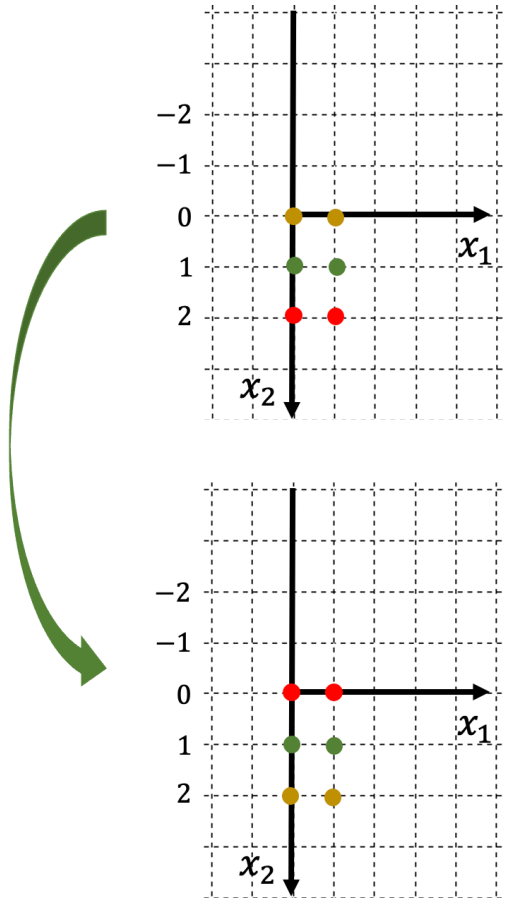
Linear Algebra and Applications

Quang-Vinh Dinh
PhD in Computer Science

Year 2024

Objectives

Matrix Transformation



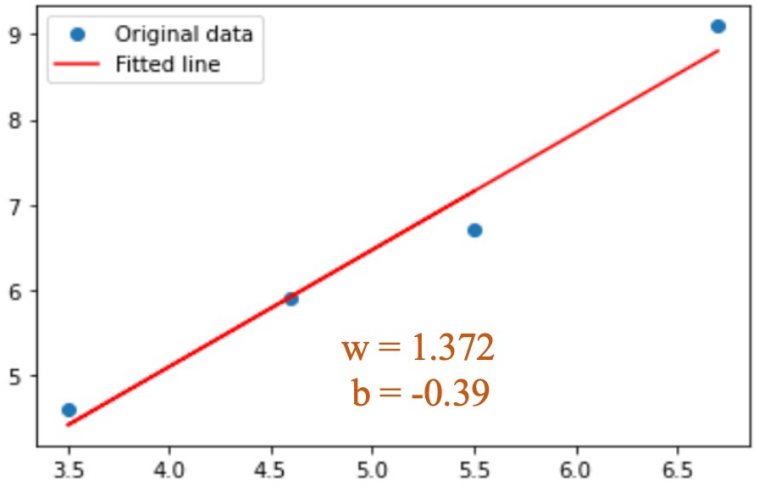
Lease-Squared Estimation

$$Ax = y$$

$$A^T Ax = A^T y$$

$$x = (A^T A)^{-1} A^T y$$

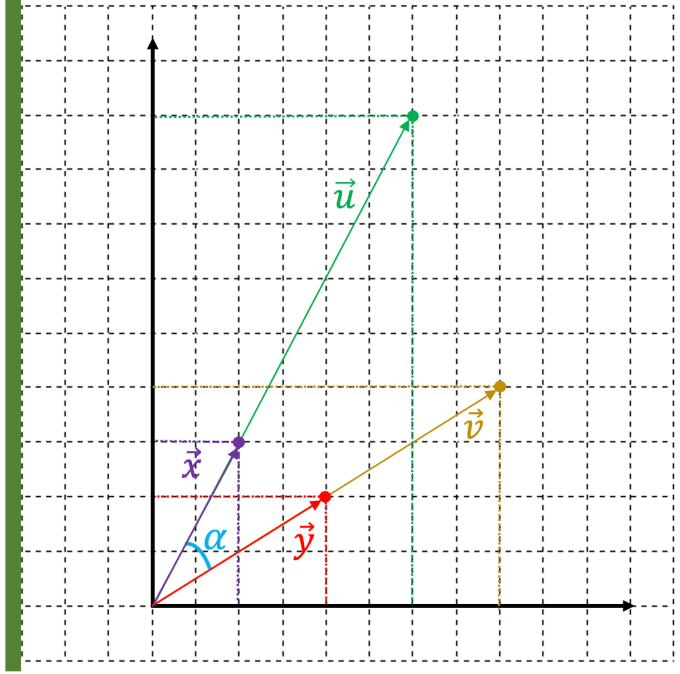
Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7



Cosine Similarity

$\vec{x} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$ $\quad \vec{u} = 3 * \vec{x} = \begin{pmatrix} 6 \\ 9 \end{pmatrix}$

$\vec{y} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$ $\quad \vec{v} = 2 * \vec{y} = \begin{pmatrix} 8 \\ 4 \end{pmatrix}$



Outline

SECTION 1

Matrix Transformation

SECTION 2

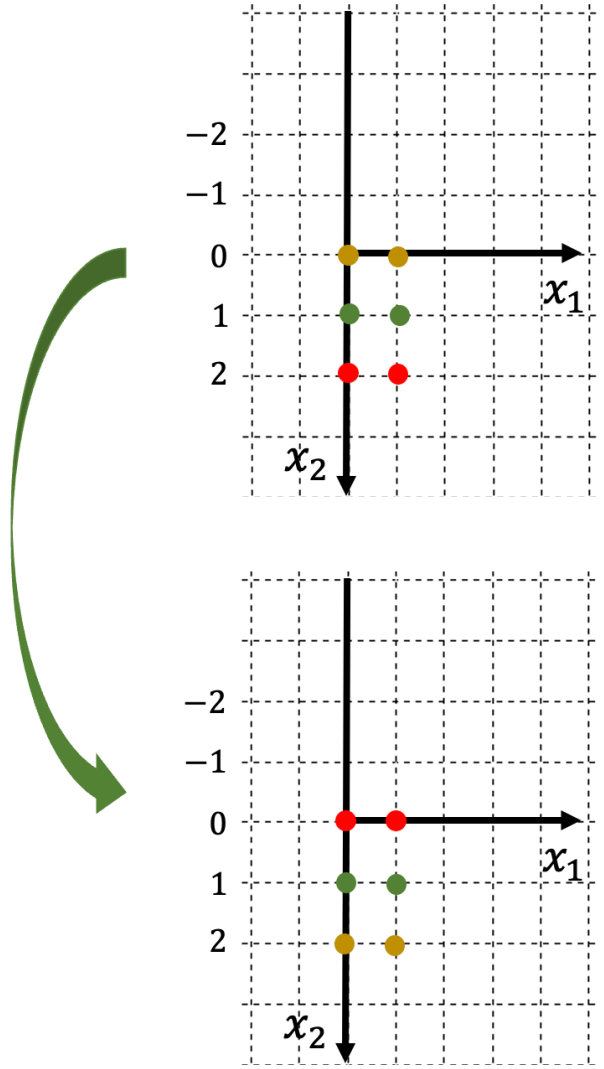
Least-squares Estimation

SECTION 3

Cosine Similarity

SECTION 4

Applications



Matrix Transformation

❖ Definition

Vector-matrix multiplication

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \qquad \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \qquad \boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

$$A\boldsymbol{x} = \boldsymbol{b}$$

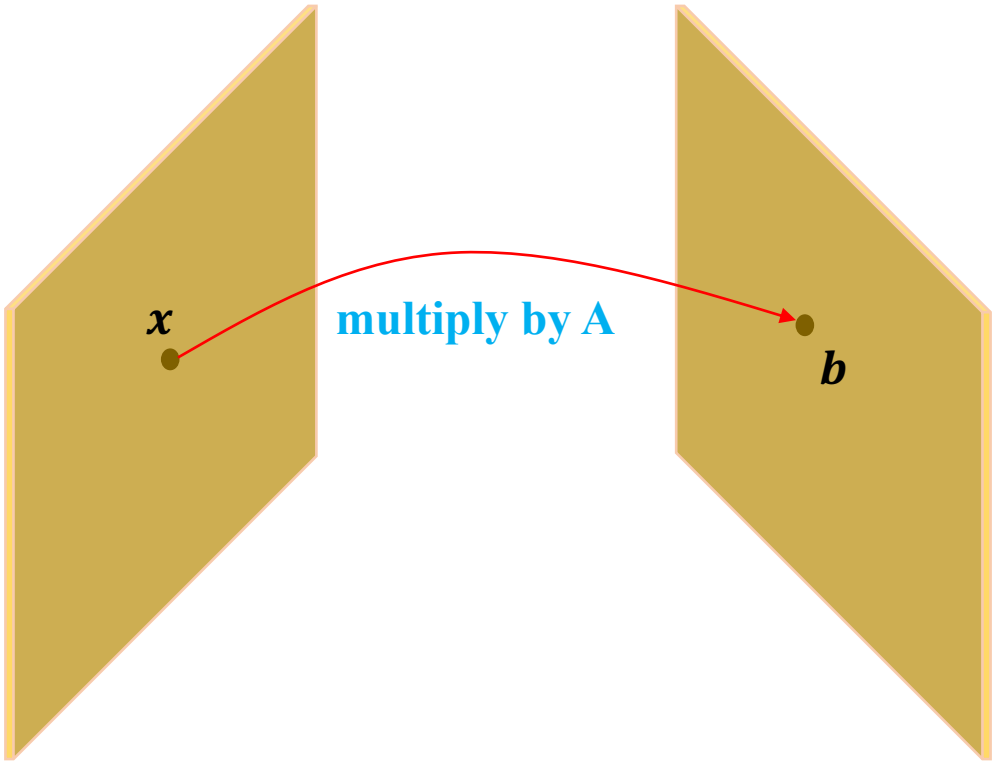
$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 \\ a_{21}x_1 + a_{22}x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Matrix **A** transforms **x** to **b**

Each element of **b** is a linear combination of all the elements of **x**

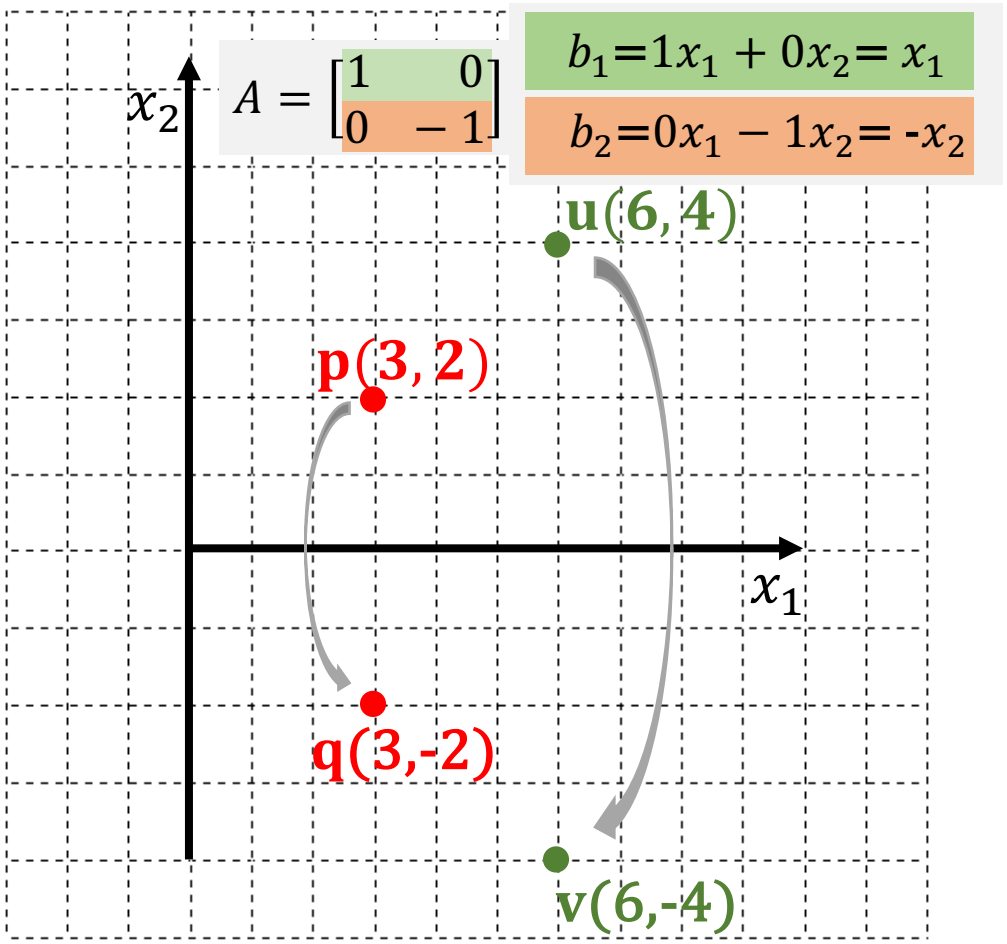
$$b_1 = a_{11}x_1 + a_{12}x_2$$

$$b_2 = a_{21}x_1 + a_{22}x_2$$



Matrix Transformation

Matrix **A** translates $\boldsymbol{x}$ symmetrically with respect to x_1

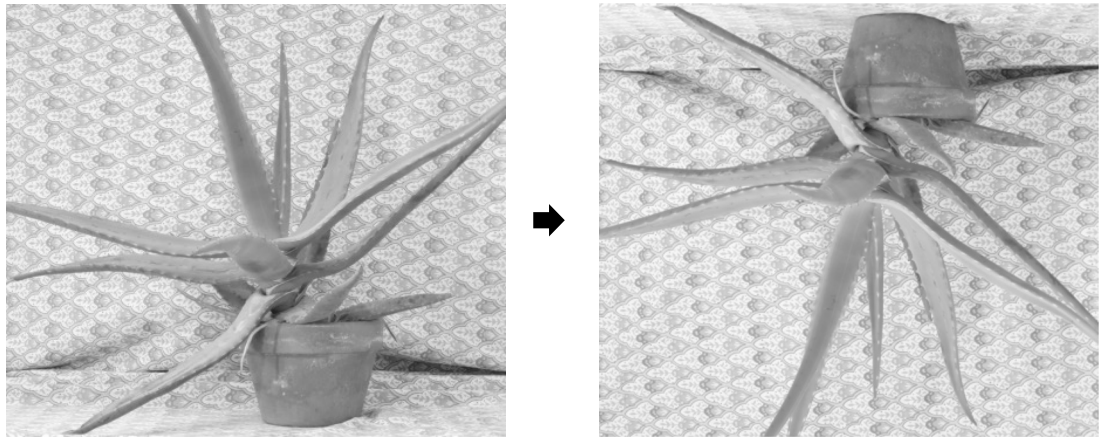


Flipping an image vertically

All the element of the pixel p(x_1, x_2, c)

coordinate of **p**

Translate a pixel (x_1, x_2) by $A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

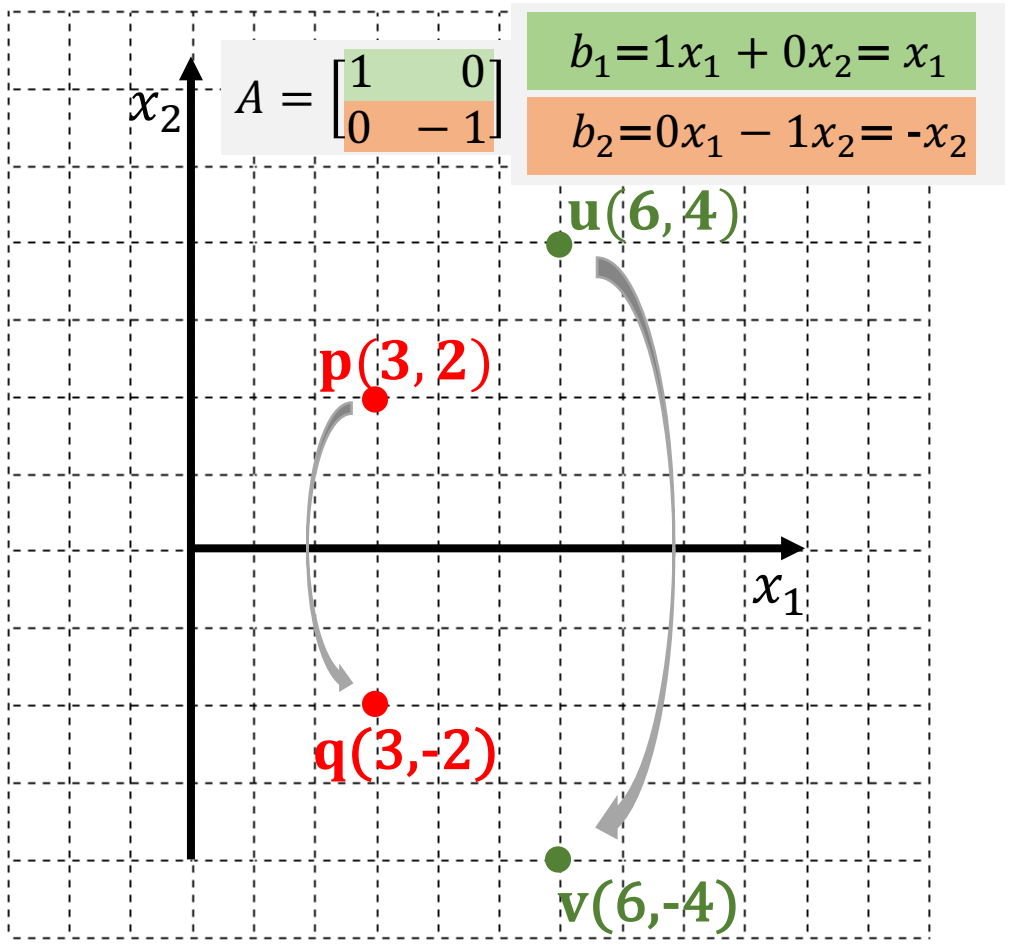


Original Image

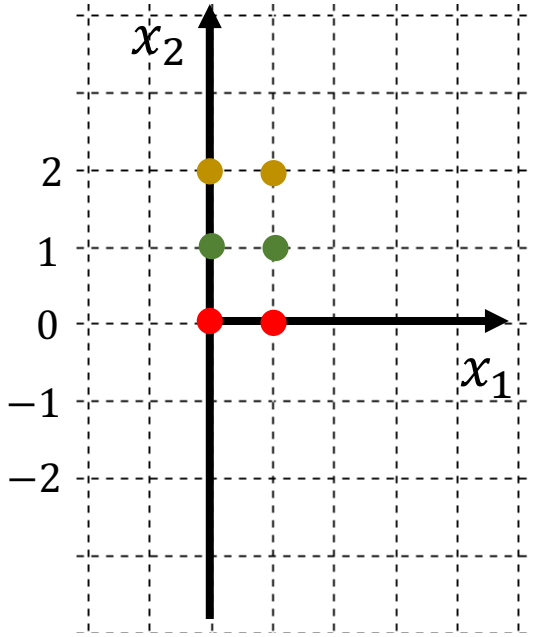
Output Image

Matrix Transformation

Matrix **A** translates $\boldsymbol{x}$ symmetrically with respect to x_1



$A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$



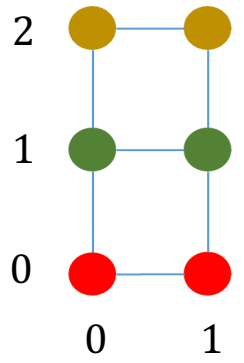
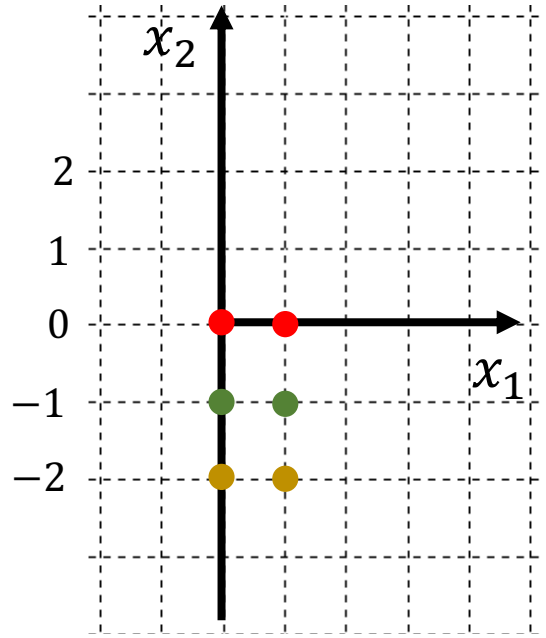
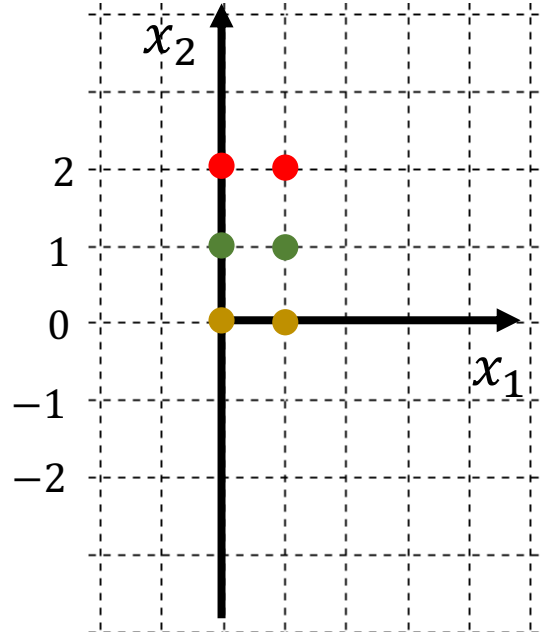
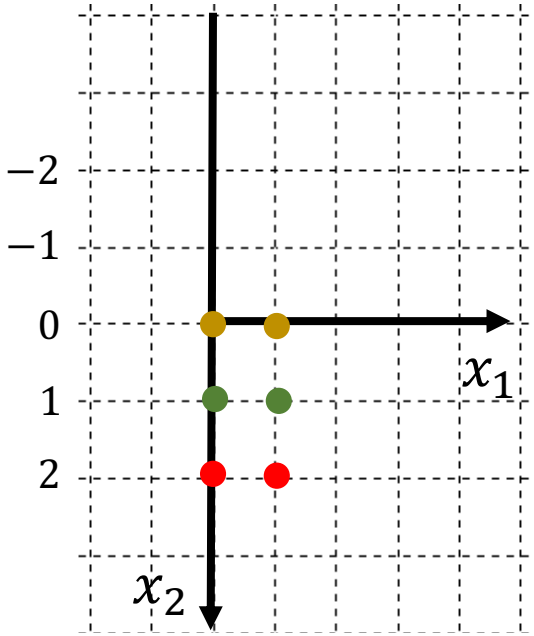


Image A

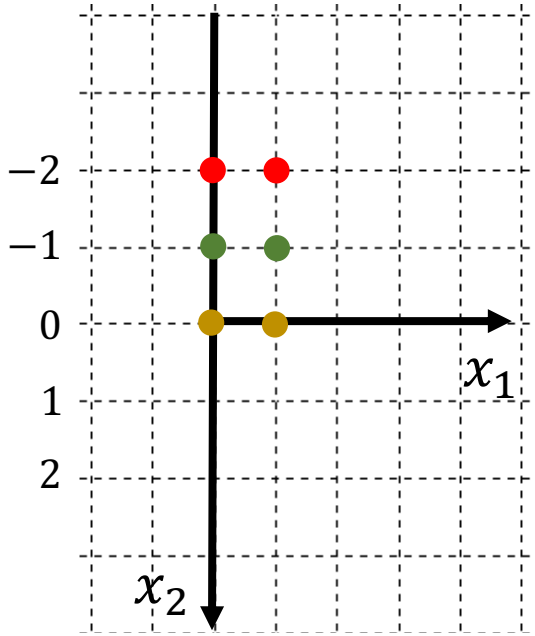


$P = \begin{bmatrix} 0 \\ H & -1 \end{bmatrix}$

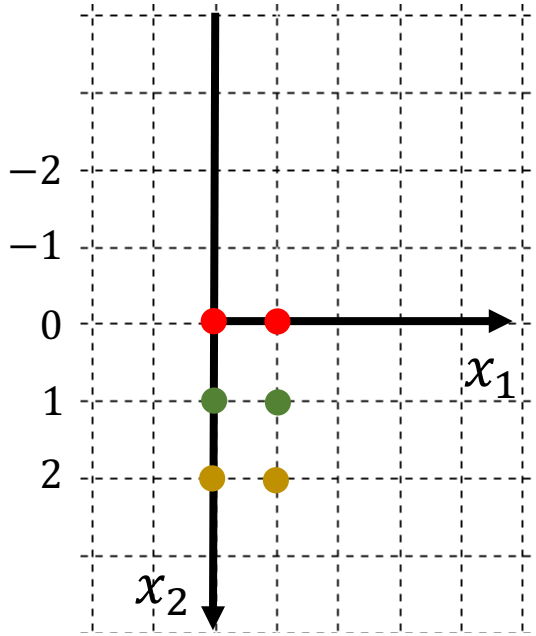




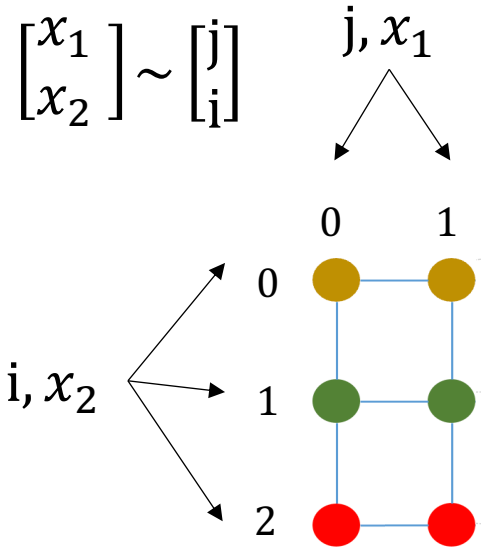
$$A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$



$$P = \begin{bmatrix} 0 \\ H-1 \end{bmatrix}$$



$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim \begin{bmatrix} j \\ i \end{bmatrix}$$



$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

```
height, width = 3, 2
img = np.arange(6).reshape(3, 2)
print(img)
```

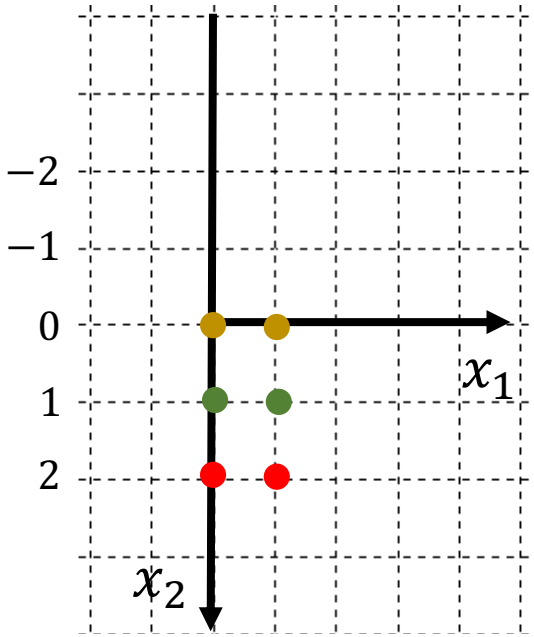
✓ 0.2s

```
[[0 1]
 [2 3]
 [4 5]]
```

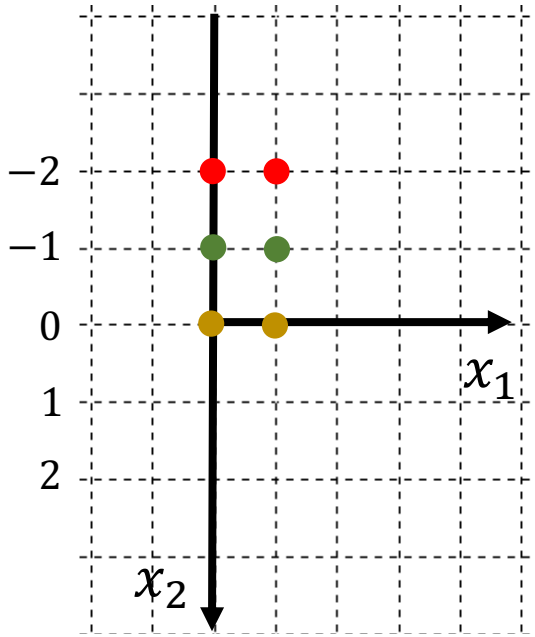
```
transform = np.array([[1, 0],
                      [0, -1]])
print(transform)
```

✓ 0.0s

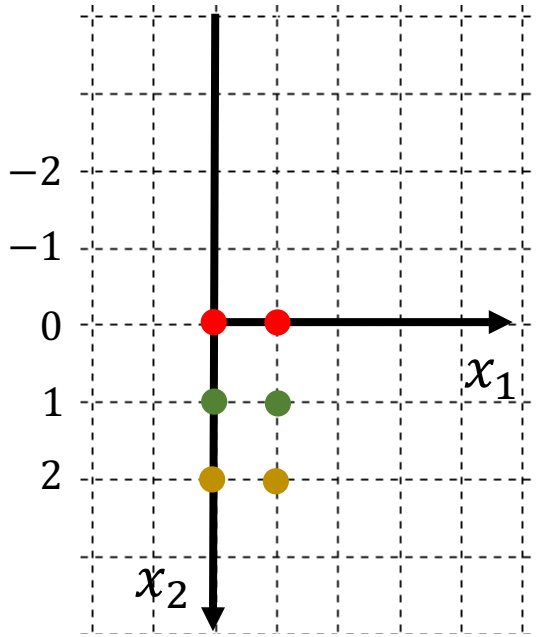
```
[[ 1  0]
 [ 0 -1]]
```



$$A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$



$$P = \begin{bmatrix} 0 \\ H-1 \end{bmatrix}$$



$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \sim \begin{bmatrix} j \\ i \end{bmatrix}$$

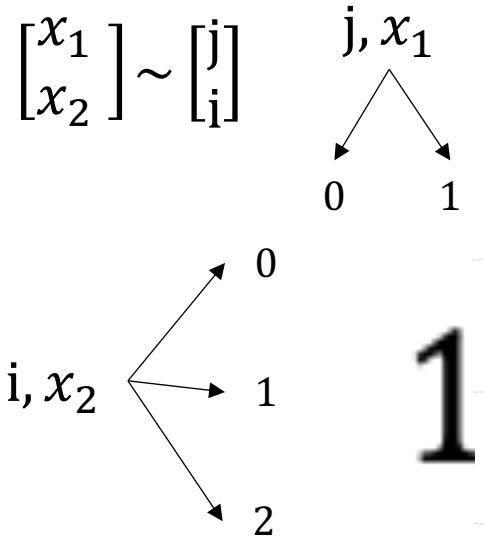


Image A

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \qquad \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \qquad \begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

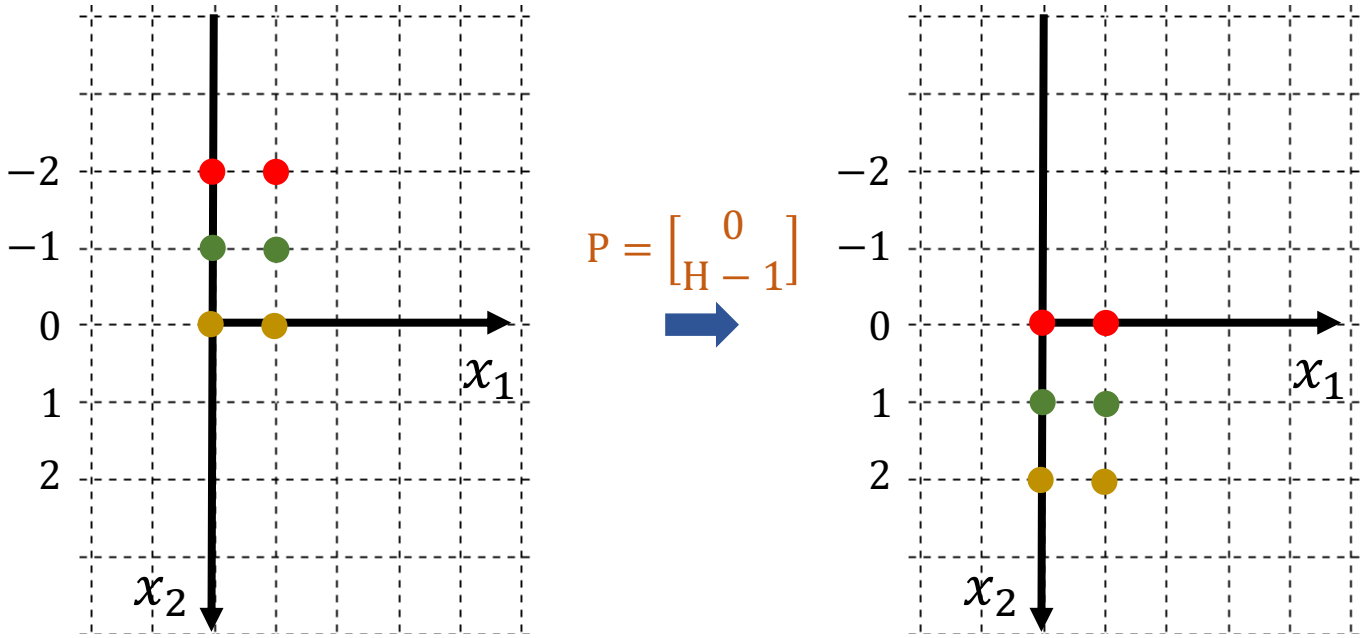
$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \qquad \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

```
A = np.array([[1, 0],
              [0, -1]])

v1 = np.array([1, 0])
print(A.dot(v1))

v2 = np.array([1, 1])
print(A.dot(v2))

v3 = np.array([1, 2])
print(A.dot(v3))
```

✓ 0.0s

```
[1 0]
[ 1 -1]
[ 1 -2]
```

Matrix Transformation



$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \rightarrow \quad \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

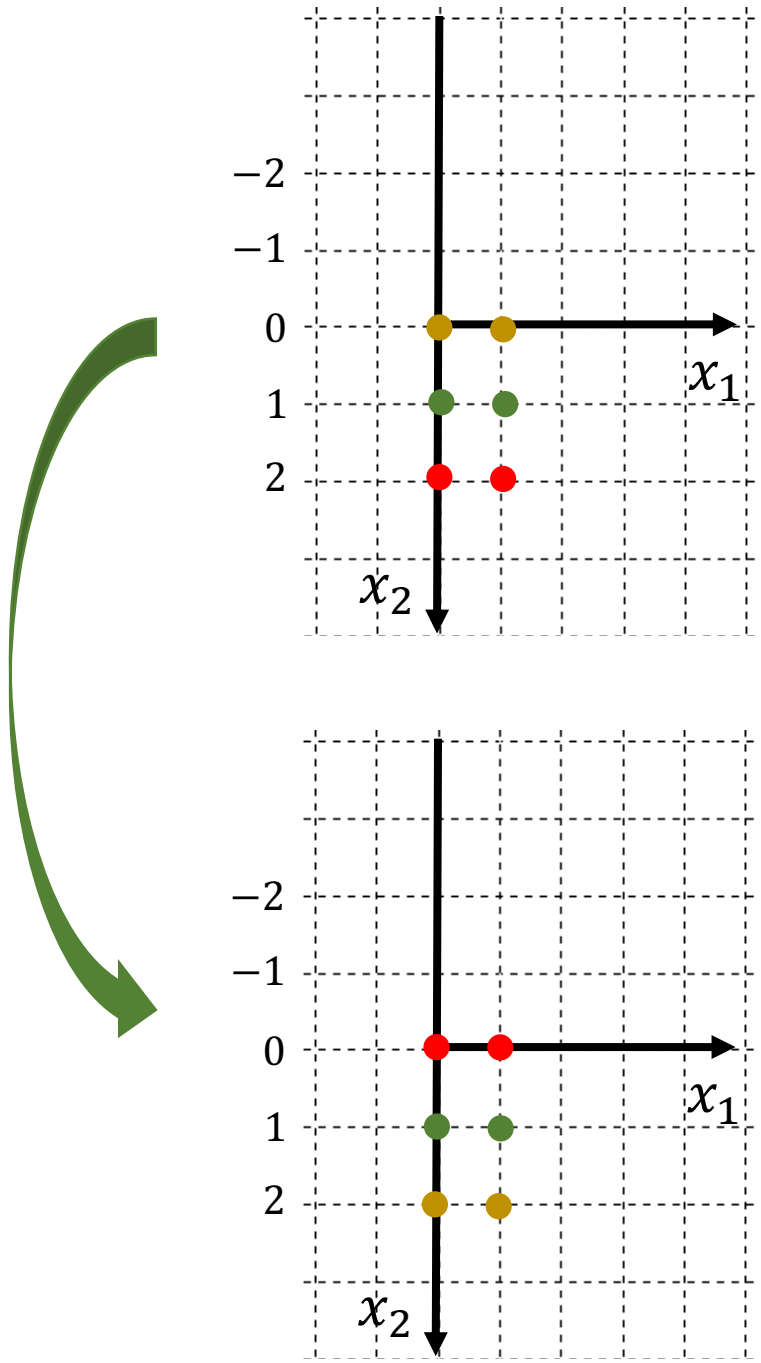
$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad \rightarrow \quad \begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \quad \rightarrow \quad \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

```python
A = np.array([[1, 0],
              [0, -1]])

H = 3
p = np.array([0, H-1])

v1 = np.array([1, 0])
print(A.dot(v1) + p)

v2 = np.array([1, 1])
print(A.dot(v2) + p)

v3 = np.array([1, 2])
print(A.dot(v3) + p)
```

✓ 0.0s

```
[1 2]
[1 1]
[1 0]
```

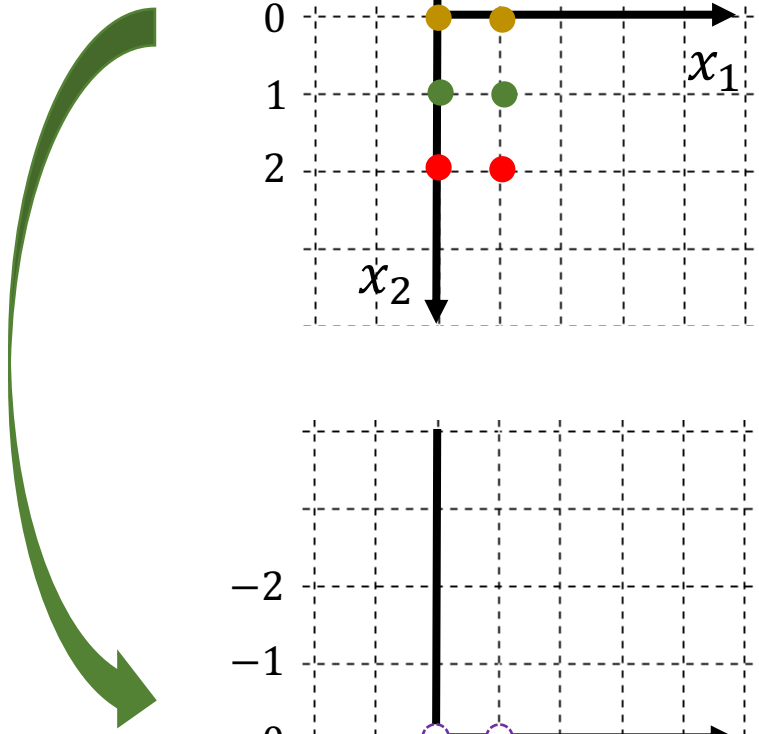



```python
# implement naively for the sake of simplicity
height = 3
width = 2
img = np.arange(6).reshape(3, 2)

A = np.array([[1, 0],
              [0, -1]])
p = np.array([0, H-1])

for x2 in range(height):
    for x1 in range(width):
        v = np.array([x1, x2])
        new_x1, new_x2 = A.dot(v) + p
        print(f'({x1, x2}) -> ({new_x1}, {new_x2})')
    print()
```

✓ 0.0s

```
((0, 0)) -> (0, 2)
((1, 0)) -> (1, 2)

((0, 1)) -> (0, 1)
((1, 1)) -> (1, 1)

((0, 2)) -> (0, 0)
((1, 2)) -> (1, 0)
```

a pixel has
(x_1, x_2, color)




```python
# implement naively for the sake of simplicity
height = 3
width = 2
img = np.arange(6).reshape(3, 2)

A = np.array([[1, 0],
              [0, -1]])
p = np.array([0, H-1])

for x2 in range(height):
    for x1 in range(width):
        v = np.array([x1, x2])
        new_x1, new_x2 = A.dot(v) + p
        print(f'({x1, x2}) -> ({new_x1}, {new_x2})')
    print()
```

✓ 0.0s

```
((0, 0)) -> (0, 2)
((1, 0)) -> (1, 2)

((0, 1)) -> (0, 1)
((1, 1)) -> (1, 1)

((0, 2)) -> (0, 0)
((1, 2)) -> (1, 0)
```

a pixel has (x_1, x_2, color)

```
height, width = 3, 2
img = np.arange(6).reshape(3, 2)
print(img)
```

✓ 0.2s

```
[[0 1]
 [2 3]
 [4 5]]
```

```
transform = np.array([[1, 0],
                      [0, -1]])
print(transform)
```

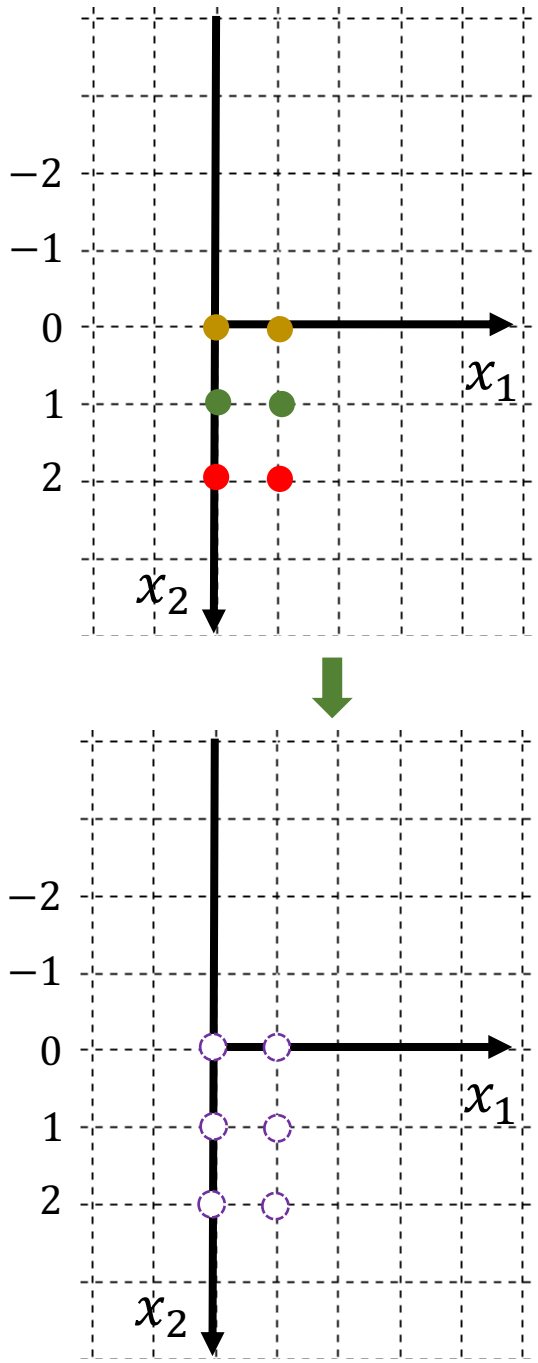
✓ 0.0s

```
[[ 1  0]
 [ 0 -1]]
```

```
output = np.zeros((height, width))
print(output)
```

✓ 0.0s

```
[[0. 0.]
 [0. 0.]
 [0. 0.]]
```




```
# implement naively for the sake of simplicity
height = 3
width = 2
img = np.arange(6).reshape(3, 2)

A = np.array([[1, 0],
              [0, -1]])
p = np.array([0, H-1])

for x2 in range(height):
    for x1 in range(width):
        v = np.array([x1, x2])
        color = img[x2, x1]

        new_x1, new_x2 = A.dot(v) + p
        output[new_x2, new_x1] = color

# print output
print(output)
```

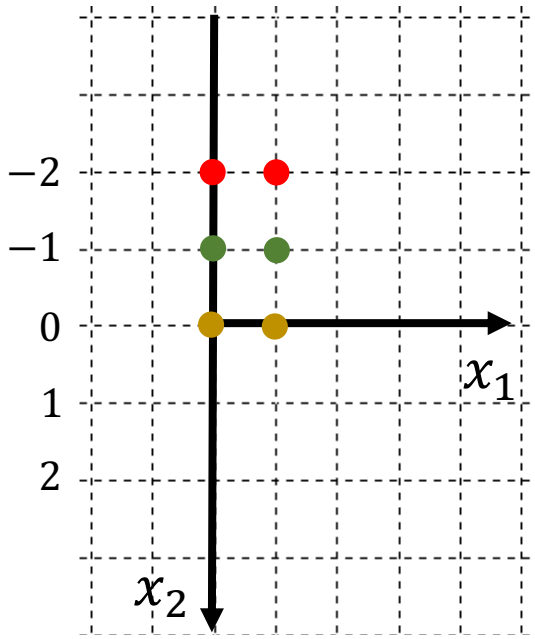
✓ 0.0s

```
[[4. 5.]
 [2. 3.]
 [0. 1.]]
```

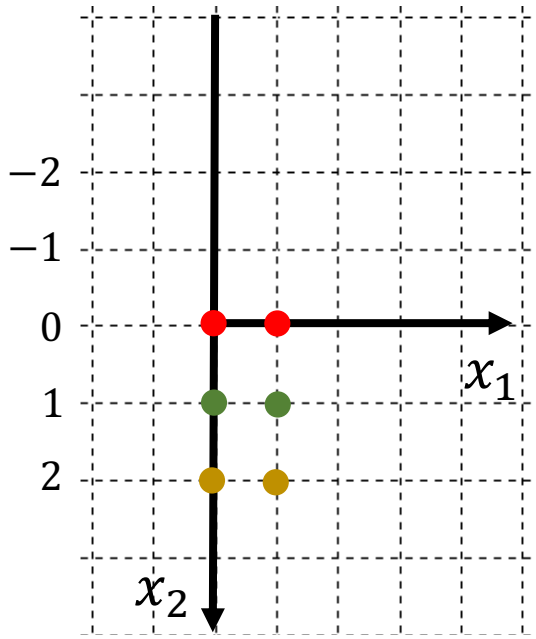
For Grayscale Images

```python
import cv2
import numpy as np

img = cv2.imread('nature_gray.png', 0)
height, width = img.shape

A = np.array([[1, 0],
              [0, -1]])
p = np.array([0, height-1])

output = np.zeros((height, width))
for x2 in range(height):
    for x1 in range(width):
        color = img[x2, x1]
        v = np.array([x1, x2])

        new_x1, new_x2 = A.dot(v) + p
        output[new_x2, new_x1] = color

cv2.imwrite('nature_gray_x.png', output)
```

Removing FORs



$$P = \begin{bmatrix} 0 \\ H-1 \end{bmatrix}$$



$$A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ H-1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

```
p = np.array([0, H-1])

v1 = np.array([1, 0])
print(A.dot(v1) + p)

v2 = np.array([1, 1])
print(A.dot(v2) + p)

v3 = np.array([1, 2])
print(A.dot(v3) + p)
```

✓ 0.0s

```
[1 2]
[1 1]
[1 0]
```

```
A = np.array([[1, 0],
              [0, -1]])

vectors = np.array([[1, 1, 1],
                    [0, 1, 2]])
print(A.dot(vectors))

H = 3
p = np.array([0, H-1]).reshape(2, 1)
print(A.dot(vectors) + p)
```

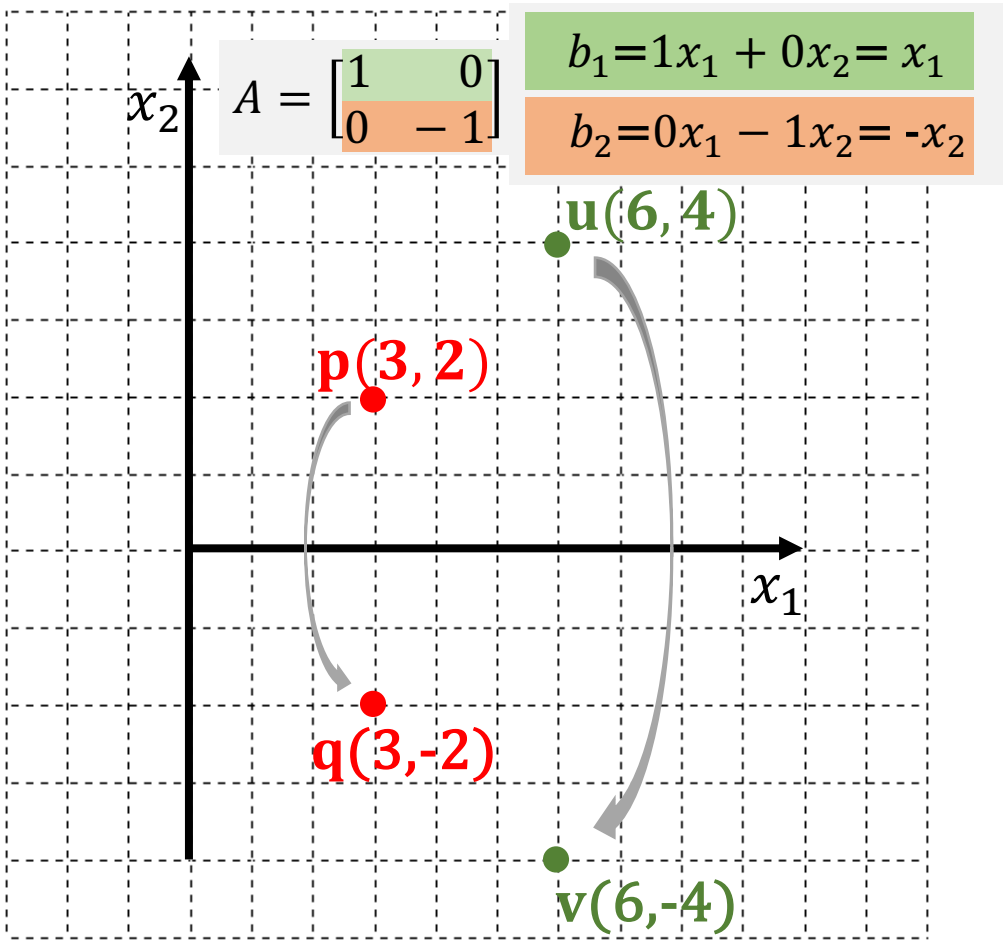
✓ 0.0s

```
[[ 1  1  1]
 [ 0 -1 -2]]
[[1 1 1]
 [2 1 0]]
```

Optional Homework

For Color Images

Matrix **A** translates $\boldsymbol{x}$ symmetrically with respect to x_1



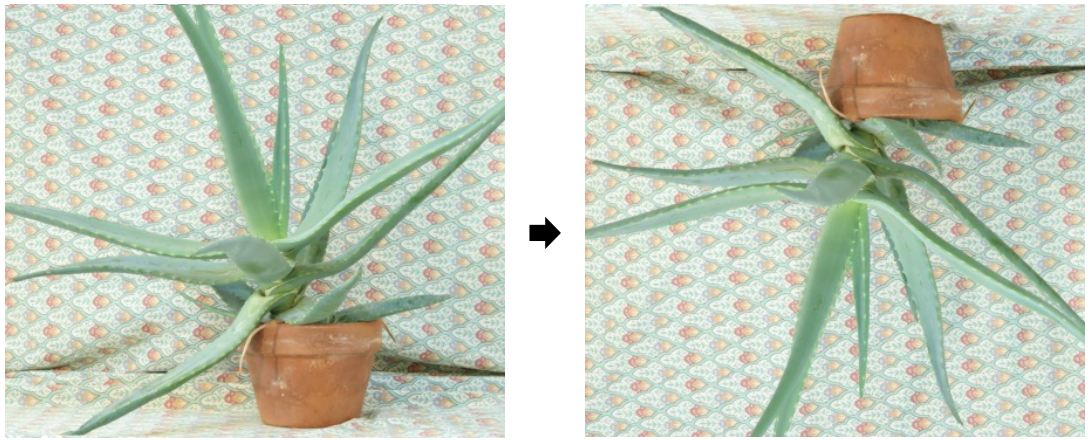
Flipping an image vertically

values (red, green, blue) of the pixel **p**

All the element of the pixel p$(x_1, x_2,$ r, g, b$)$

coordinate of **p**

Translate a pixel (x_1, x_2) **by** $A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$



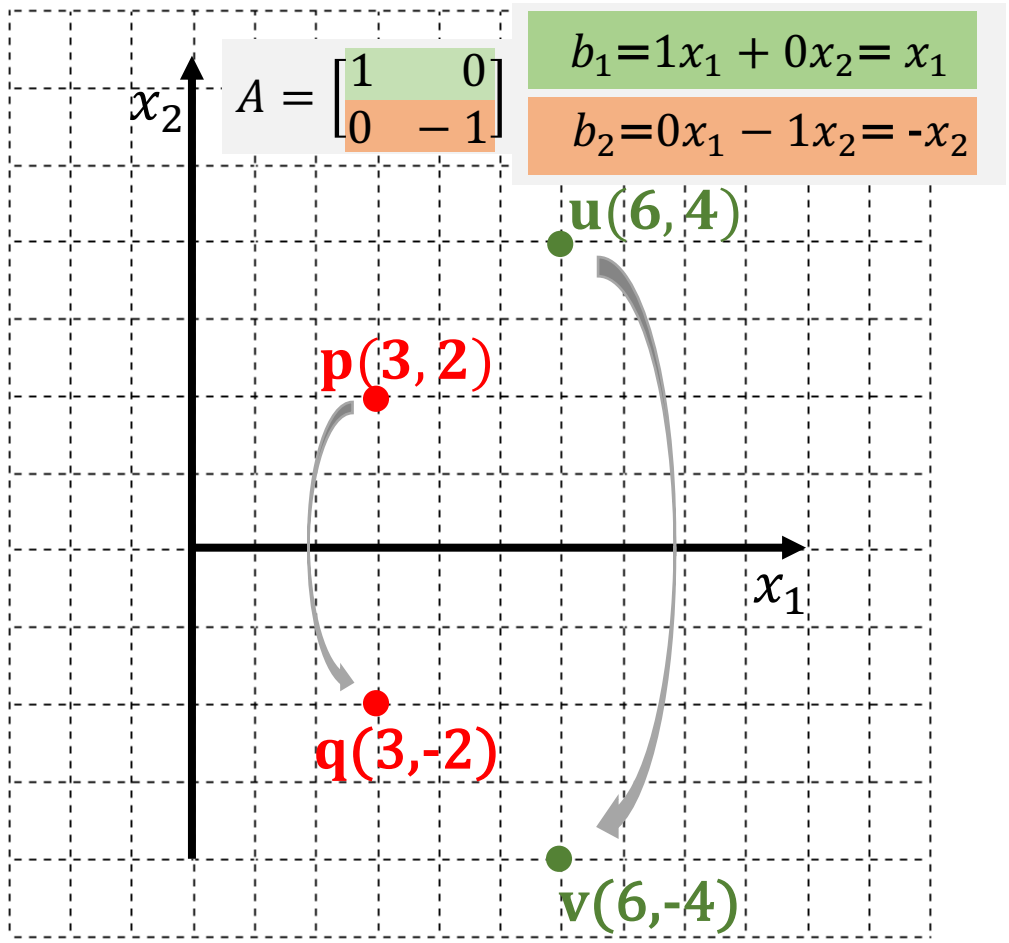
Original Image

Output Image

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('nature.jpg', 1)
img = img.astype(float)
height, width, depth = img.shape

transform = np.array([[1, 0],
                      [0, -1]])

output = np.zeros((height, width, depth))
for i in range(height):
    for j in range(width):
        pixel = img[i, j, :]

        new_j, new_i = transform.dot(np.array([j, i]))
                       + [0, height-1]
        output[new_i, new_j, :] = pixel

output = output.astype(np.uint8)
```

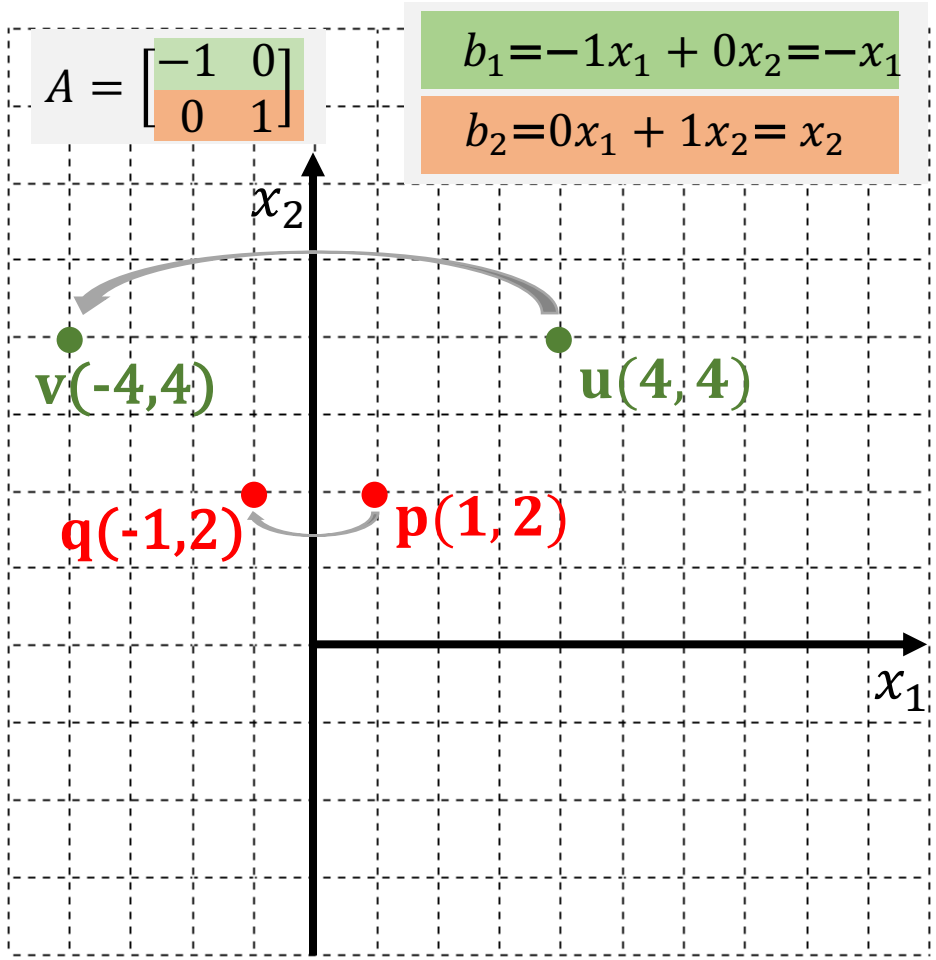
For Color Images

Matrix **A** translates $\boldsymbol{x}$ symmetrically with respect to x_1



Flipping Horizontally

Matrix **A** translates $\boldsymbol{x}$ symmetrically with respect to x_2



Flipping an image horizontally

values (red, green, blue) of the pixel **p**

All the element of the pixel p$(x_1, x_2,$ r, g, b$)$

coordinate of **p**

Translate a pixel (x_1, x_2) **by** $A = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$

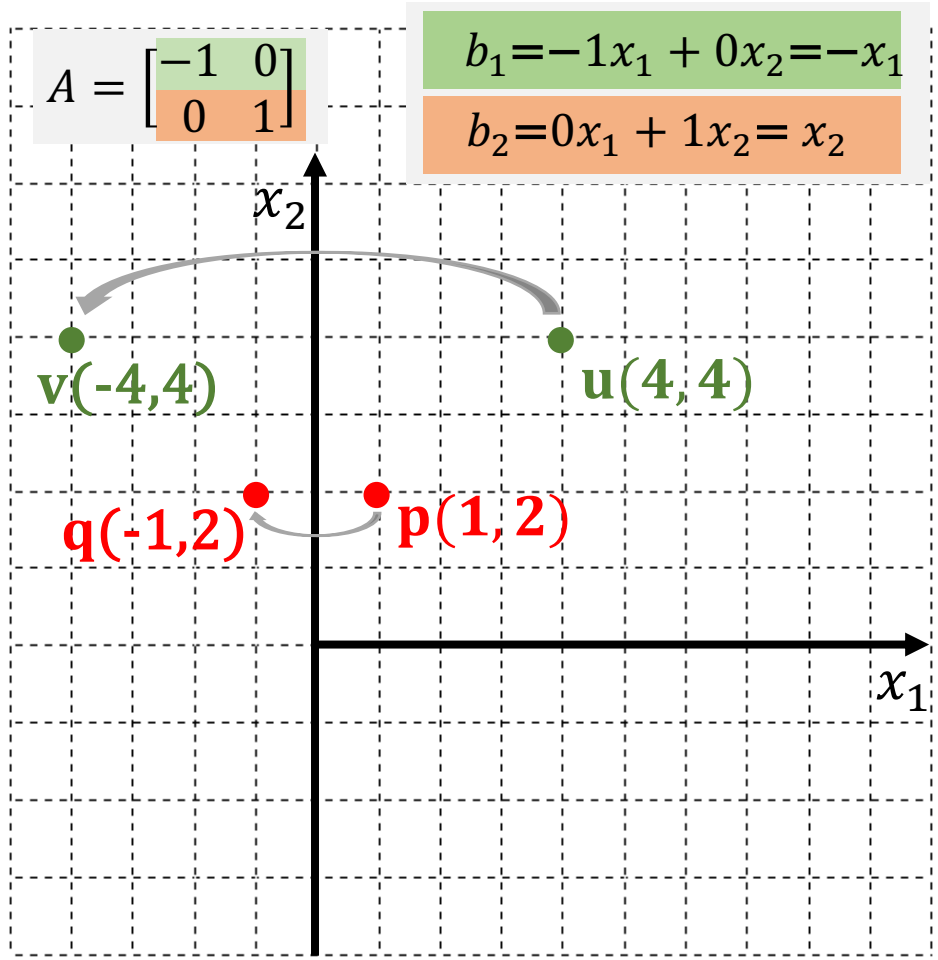
Original Image

Output Image

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('nature.jpg', 1)
img = img.astype(float)
height, width, depth = img.shape

transform = np.array([[-1, 0],
                      [0, 1]])

output = np.zeros((height, width, depth))
for i in range(height):
    for j in range(width):
        pixel = img[i, j, :]
        new_j, new_i = transform.dot(np.array([j, i]))
                       + [width-1, 0]
        output[new_i, new_j, :] = pixel

output = output.astype(np.uint8)
cv2.imwrite('flipping_y.jpg', output)
```

Flipping Horizontally

Matrix **A** translates $\boldsymbol{x}$ symmetrically with respect to x_2



Outline

SECTION 1

Matrix Transformation

SECTION 2

Least-squares Estimation

SECTION 3

Cosine Similarity

SECTION 4

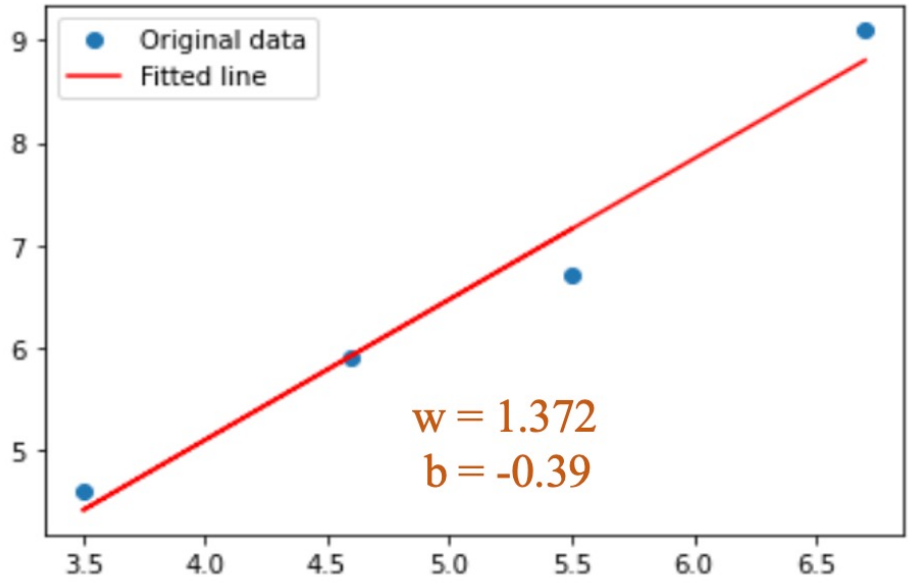
Applications

$$Ax = y$$

$$A^T Ax = A^T y$$

$$x = (A^T A)^{-1} A^T y$$

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7



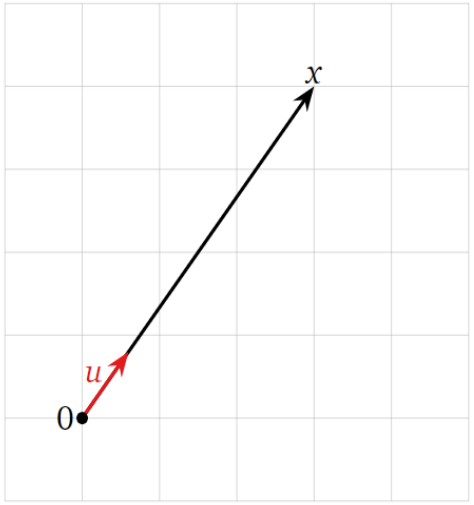
Least Squares Estimation

❖ Distance

What is the unit vector u in the direction of $x = \begin{pmatrix}3\\4\end{pmatrix}$?

Solution
We divide by the length of x:

$$u = \frac{x}{\|x\|} = \frac{1}{\sqrt{3^2+4^2}}\begin{pmatrix}3\\4\end{pmatrix} = \frac{1}{5}\begin{pmatrix}3\\4\end{pmatrix} = \begin{pmatrix}3/5\\4/5\end{pmatrix}$$



Definition. The ***distance*** between two points x, y in $\mathbf{R}^n$ is the length of the vector from x to y:

$$\text{dist}(x, y) = \|y - x\|$$

Definition. A ***unit vector*** is a vector x with length $\|x\| = \sqrt{x \cdot x} = 1$

Let x be a nonzero vector in $\mathbf{R}^n$. *The* ***unit vector in the direction of*** *x is the vector* $x/\|x\|$

https://textbooks.math.gatech.edu/ila/index.html

Least Squares Estimation

❖ Matrix inverses

How to 'divide' by a matrix?

$$Ax = b \iff x = A^{-1}b$$

The *inverse* of a nonzero number a is the number b which is characterized by the property that ab=1

Definition. Let A be an $n \times n$ (square) matrix. We say that A is ***invertible*** if there is an $n \times n$ matrix B such that

$$AB = I \quad and \quad BA = I$$

In this case, the matrix B is called the ***inverse*** of A and we write $B = A^{-1}$

Least Squares Estimation

❖ Determinant and Inverse

The ***determinant*** of a 2×2 matrix is the number

$$\det\begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

Let $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$

1.if $\det(A) \neq 0$, *then* A *is invertible , and*

$$A^{-1} = \frac{1}{\det(A)}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

2. if $\det(A) = 0$, *then* A *is not invertible*

Let

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

$\det(A) = 1 \cdot 4 - 2 \cdot 3 = -2$, the matrix A is invertible with inverse

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}^{-1} = \frac{1}{\det(A)}\begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = -\frac{1}{2}\begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix}$$

We check:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \cdot -\frac{1}{2}\begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = -\frac{1}{2}\begin{pmatrix} -2 & 0 \\ 0 & -2 \end{pmatrix} = I_2$$

Least Squares Estimation

❖ Matrix inverses

Definition. Let A be an $n \times n$ (square) matrix. We say that A is ***invertible*** if there is an $n \times n$ matrix B such that

$$AB = I \quad and \quad BA = I$$

In this case, the matrix B is called the ***inverse*** of A and we write $B = A^{-1}$

```
3  import numpy as np
4
5  matrix = np.array([[1., 2.],
6                     [4., 6.]])
7  invert = np.linalg.inv(matrix)
8
9  print('matrix\n', matrix)
10 print('Invert\n', invert)
```

```
matrix
 [[1. 2.]
 [4. 6.]]
Invert
 [[-3.   1. ]
 [ 2.  -0.5]]
```

```
1  np.dot(matrix, invert)
```

```
array([[1., 0.],
       [0., 1.]])
```

```
1  np.dot(invert, matrix)
```

```
array([[1., 0.],
       [0., 1.]])
```

https://textbooks.math.gatech.edu/ila/index.html

Least Squares Estimation

❖ Symmetric and invertible

A is symmetric if

for every i, j, $a_{ij}=a_{ji}$

$\Leftrightarrow A = A^T$

Example

$$A = \begin{bmatrix} 10 & 7.1 \\ 7.1 & 2.0 \end{bmatrix}$$

A^TA is an invertible and symmetric matrix only if A has independent columns

Independent columns

$$A = \begin{bmatrix} 6.7 & 1 \\ 4.6 & 1 \\ 3.5 & 1 \\ 5.5 & 1 \end{bmatrix} \qquad A^T = \begin{bmatrix} 6.7 & 4.6 & 3.5 & 5.5 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$A^TA = \begin{bmatrix} 6.7 & 4.6 & 3.5 & 5.5 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 6.7 & 1 \\ 4.6 & 1 \\ 3.5 & 1 \\ 5.5 & 1 \end{bmatrix} = \begin{bmatrix} 108.55 & 20.3 \\ 20.3 & 4.0 \end{bmatrix}$$

Dependent columns

$$A = \begin{bmatrix} 2 & 4 \\ 3 & 6 \\ 2 & 4 \\ 1 & 2 \end{bmatrix} \qquad A^T = \begin{bmatrix} 2 & 3 & 2 & 1 \\ 4 & 6 & 4 & 2 \end{bmatrix}$$

$$A^TA = \begin{bmatrix} 2 & 3 & 2 & 1 \\ 4 & 6 & 4 & 2 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 3 & 6 \\ 2 & 4 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 18 & 36 \\ 36 & 72 \end{bmatrix}$$

$$det(A^TA) = \begin{vmatrix} 18 & 36 \\ 36 & 72 \end{vmatrix} = 0 \qquad \Longrightarrow A^TA \text{ is non-invertible}$$

Least Squares Estimation

❖ Matrix inverses

Theorem. *Let A be an $n \times n$ matrix and let $(A \mid I_n)$ be the matrix obtained by augmenting A by the identity matrix. If the reduced row echelon form of $(A \mid I_n)$ has the form $(I_n \mid B)$, then A is invertible and $B = A^{-1}$. Otherwise, A is not invertible.*

Find the inverse of the matrix

$$A = \begin{pmatrix} 1 & 0 & 4 \\ 0 & 1 & 2 \\ 0 & -3 & -4 \end{pmatrix}$$

Solution

$$\left(\begin{array}{ccc|ccc} 1 & 0 & 4 & 1 & 0 & 0 \\ 0 & 1 & 2 & 0 & 1 & 0 \\ 0 & -3 & -4 & 0 & 0 & 1 \end{array}\right) \xrightarrow{R_3 = R_3 + 3R_2} \left(\begin{array}{ccc|ccc} 1 & 0 & 4 & 1 & 0 & 0 \\ 0 & 1 & 2 & 0 & 1 & 0 \\ 0 & 0 & 2 & 0 & 3 & 1 \end{array}\right)$$

$$\xrightarrow[R_2 = R_2 - R_3]{R_1 = R_1 - 2R_3} \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & -6 & -2 \\ 0 & 1 & 0 & 0 & -2 & -1 \\ 0 & 0 & 2 & 0 & 3 & 1 \end{array}\right)$$

$$\xrightarrow{R_3 = R_3 \div 2} \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & -6 & -2 \\ 0 & 1 & 0 & 0 & -2 & -1 \\ 0 & 0 & 1 & 0 & 3/2 & 1/2 \end{array}\right)$$

By the theorem, the inverse matrix is

$$\begin{pmatrix} 1 & 0 & 4 \\ 0 & 1 & 2 \\ 0 & -3 & -4 \end{pmatrix}^{-1} = \begin{pmatrix} 1 & -6 & -2 \\ 0 & -2 & -1 \\ 0 & 3/2 & 1/2 \end{pmatrix}$$

We check

$$\begin{pmatrix} 1 & 0 & 4 \\ 0 & 1 & 2 \\ 0 & -3 & -4 \end{pmatrix} \begin{pmatrix} 1 & -6 & -2 \\ 0 & -2 & -1 \\ 0 & 3/2 & 1/2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

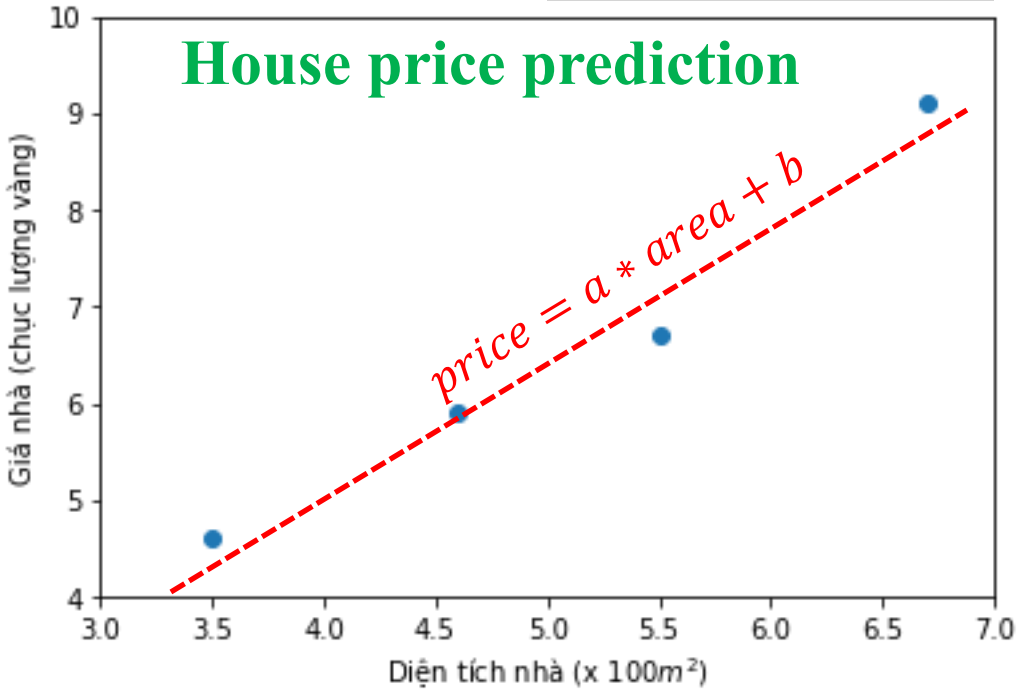
https://textbooks.math.gatech.edu/ila/index.html

Least Squares Estimation

❖ Examples

Given sample data

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7



$$price = a * area + b$$

$$9.1 = a * 6.7 + b$$
$$5.9 = a * 4.6 + b$$
$$4.6 = a * 3.5 + b$$
$$6.7 = a * 5.5 + b$$

$$\begin{bmatrix} 9.1 \\ 5.9 \\ 4.6 \\ 6.7 \end{bmatrix} = \begin{bmatrix} 6.7 & 1 \\ 4.6 & 1 \\ 3.5 & 1 \\ 5.5 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}$$

Least Squares Estimation

Given sample data

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7

symmetric and invertible

$$Ax = y$$

$$A^T Ax = A^T y$$

$$x = (A^T A)^{-1} A^T y$$

*The columns of A are linearly independent

$$A = \begin{bmatrix} 6.7 & 1 \\ 4.6 & 1 \\ 3.5 & 1 \\ 5.5 & 1 \end{bmatrix} \qquad y = \begin{bmatrix} 9.1 \\ 5.9 \\ 4.6 \\ 6.7 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 6.7 & 4.6 & 3.5 & 5.5 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$A^T A = \begin{bmatrix} 6.7 & 4.6 & 3.5 & 5.5 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 6.7 & 1 \\ 4.6 & 1 \\ 3.5 & 1 \\ 5.5 & 1 \end{bmatrix} = \begin{bmatrix} 108.55 & 20.3 \\ 20.3 & 4.0 \end{bmatrix}$$

$$det(A^T A) = \begin{vmatrix} 108.55 & 20.3 \\ 20.3 & 4.0 \end{vmatrix} = 22.11 \Rightarrow det(A^T A) \neq 0$$

$$B = \begin{bmatrix} b_1 & b_2 \\ b_3 & b_4 \end{bmatrix} \qquad B^{-1} = \frac{1}{det(B)} \begin{bmatrix} b_4 & -b_2 \\ -b_3 & b_1 \end{bmatrix}$$

$$(A^T A)^{-1} = \frac{1}{det(A^T A)} \begin{bmatrix} 4.0 & -20.3 \\ -20.3 & 108.55 \end{bmatrix} = \begin{bmatrix} 0.18 & -0.92 \\ -0.92 & 4.91 \end{bmatrix}$$

$$x = (A^T A)^{-1} A^T y = \begin{bmatrix} 0.18 & -0.92 \\ -0.92 & 4.91 \end{bmatrix} \begin{bmatrix} 6.7 & 4.6 & 3.5 & 5.5 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 9.1 \\ 5.9 \\ 4.6 \\ 6.7 \end{bmatrix}$$

$$= \begin{bmatrix} 0.18 & -0.92 \\ -0.92 & 4.91 \end{bmatrix} \begin{bmatrix} 141.06 \\ 26.3 \end{bmatrix} = \begin{bmatrix} 1.3726 \\ -0.391 \end{bmatrix}$$

Least Squares Estimation

❖ Examples

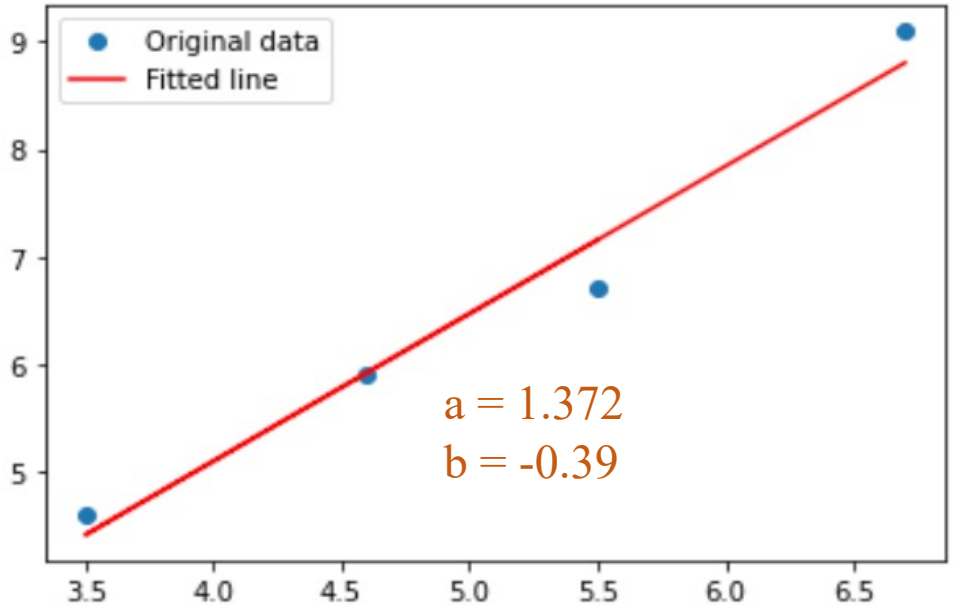
Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7

*The columns of A are linearly independent

$$Ax = y$$

$$A^T Ax = A^T y$$

$$x = (A^T A)^{-1} A^T y$$




```python
# step-by-step least squares estimation
import numpy as np

A = np.array([[6.7, 1],
              [4.6, 1],
              [3.5, 1],
              [5.5, 1]])

y = np.array([9.1, 5.9, 4.6, 6.7])

# compute A(^T)A
ATA = np.dot(A.T, A)

# compute determinant
d_value = np.linalg.det(ATA)

# compute the inverse of a matrix
ATAminus1 = np.linalg.inv(ATA)

# compute lse
x = np.dot(ATAminus1, np.dot(A.T, y))
print(x)
```

```
[ 1.37268204 -0.39136137]
```

Least Squares Estimation

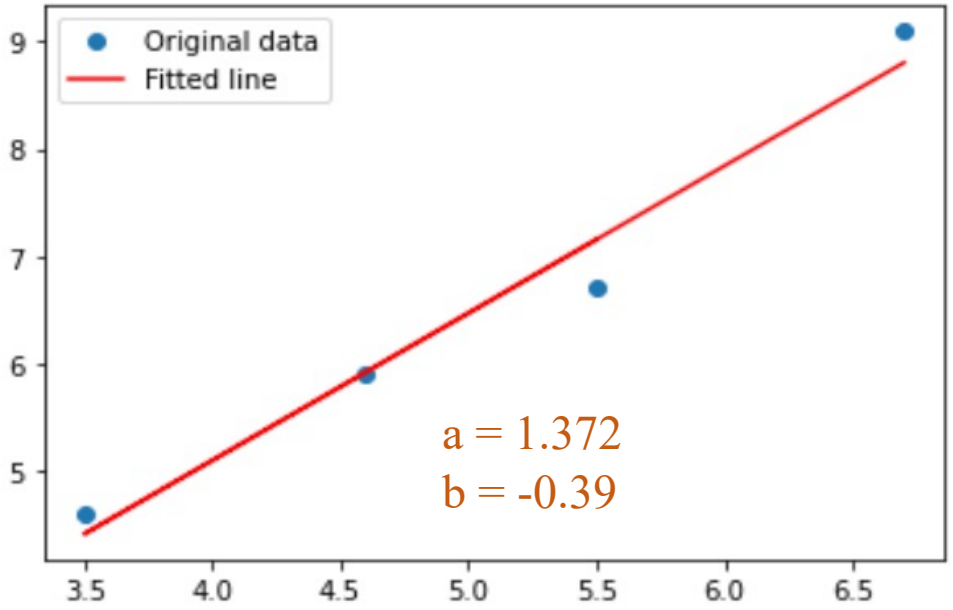
❖ Examples

$$Ax = y$$

$$A^T Ax = A^T y$$

$$x = (A^T A)^{-1} A^T y$$

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7




```python
# using np.linalg.lstsq()
import numpy as np

# data
x = np.array([6.7, 4.6, 3.5, 5.5])
y = np.array([9.1, 5.9, 4.6, 6.7])

# create A = [x 1]
A = np.vstack([x, np.ones(len(x))]).T

# compute least square
w, b = np.linalg.lstsq(A, y, rcond=None)[0]
print(w, b)
```

```
1.3726820443238348 -0.39136137494346346
```

Outline

SECTION 1

Matrix Transformation

SECTION 2

Least-squares Estimation

SECTION 3

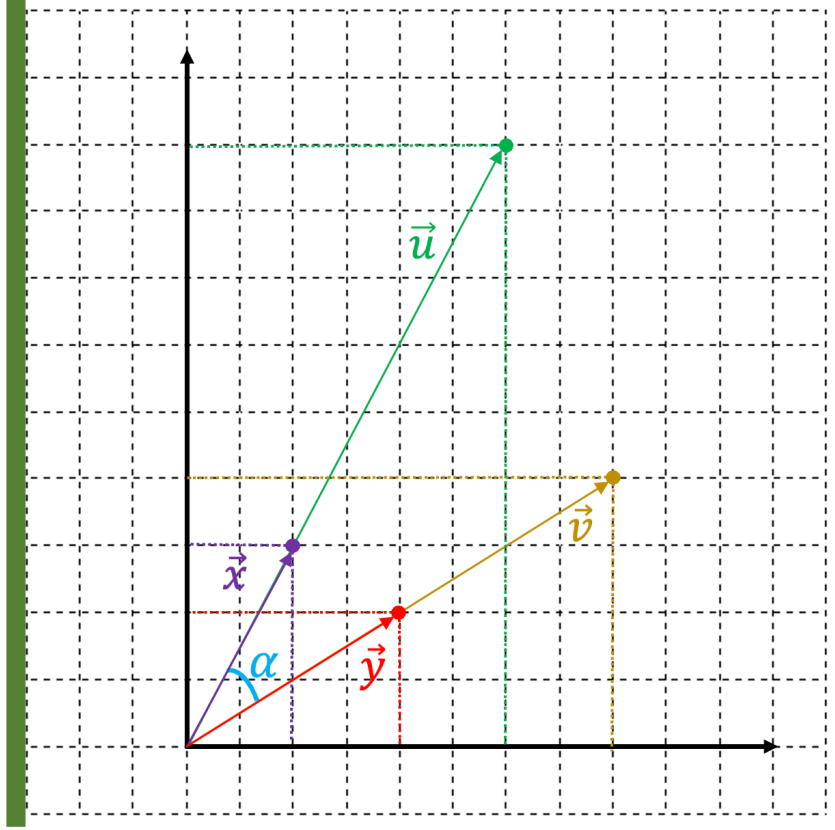
Cosine Similarity

SECTION 4

Applications

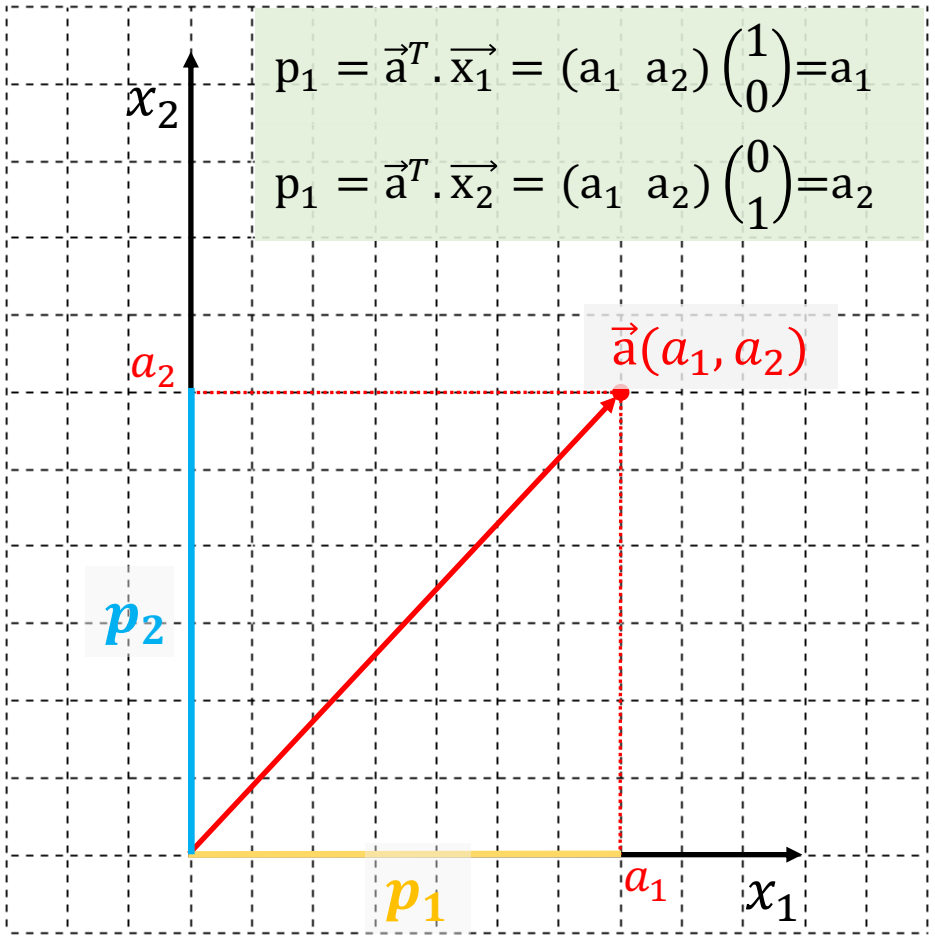
$\vec{x} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$ $\quad \vec{u} = 3 * \vec{x} = \begin{pmatrix} 6 \\ 9 \end{pmatrix}$

$\vec{y} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$ $\quad \vec{v} = 2 * \vec{y} = \begin{pmatrix} 8 \\ 4 \end{pmatrix}$



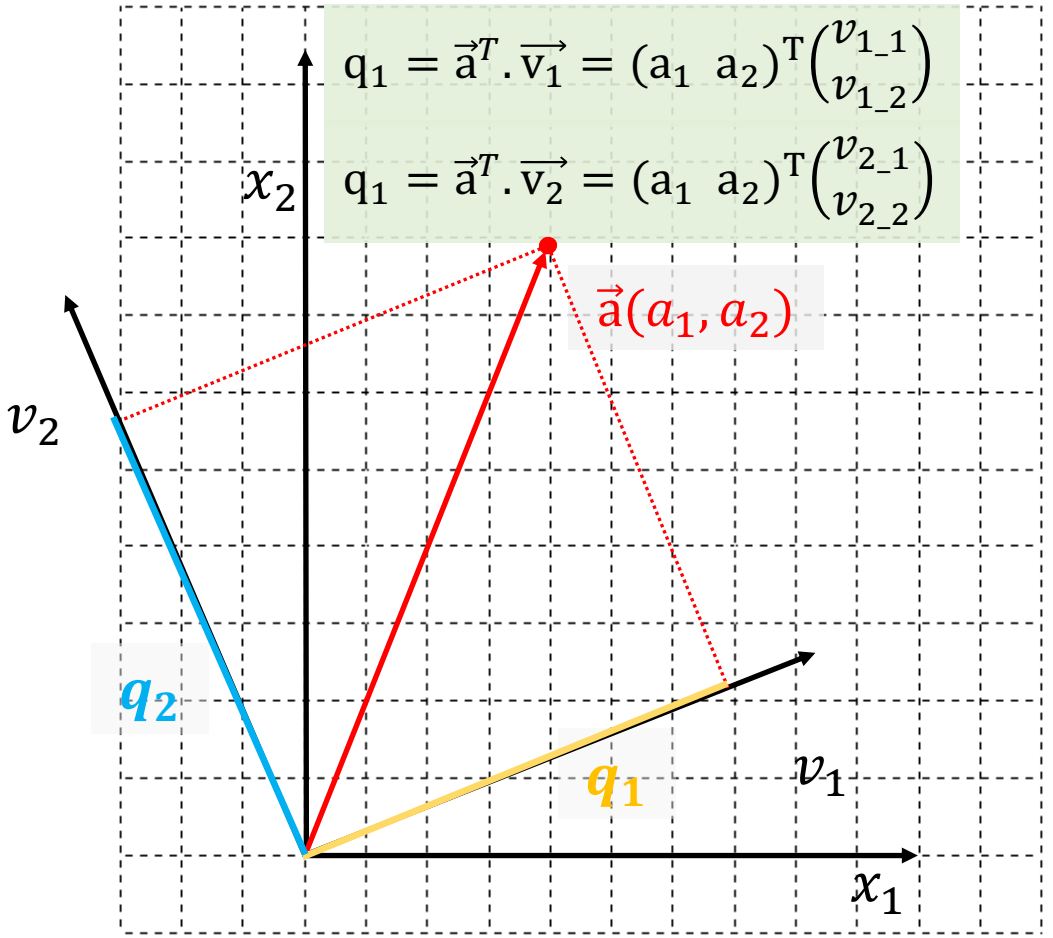
Dot Product

$\overrightarrow{x_1} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ $\overrightarrow{x_2} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$



Tìm độ dài hình chiếu của $\vec{a}$ lên $\overrightarrow{x_1}$ và $\overrightarrow{x_2}$

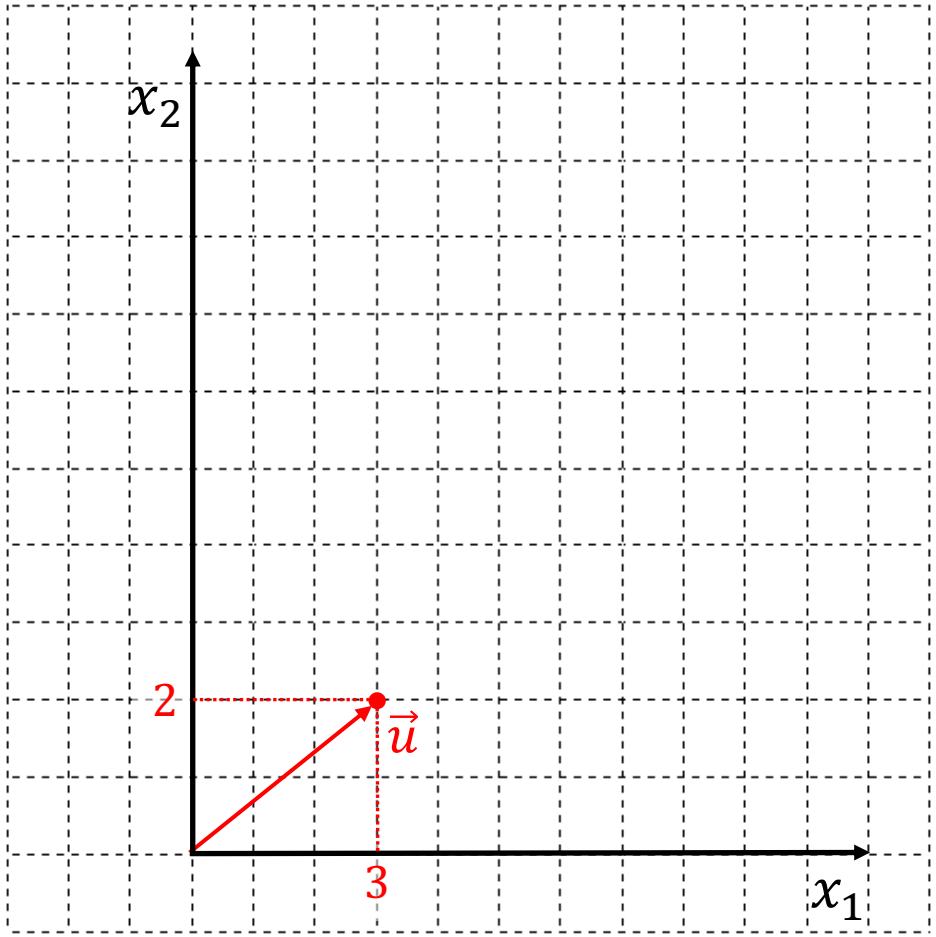
$\overrightarrow{v_1} = \begin{pmatrix} v_{1\_1} \\ v_{1\_2} \end{pmatrix}$ $\overrightarrow{v_2} = \begin{pmatrix} v_{2\_1} \\ v_{2\_2} \end{pmatrix}$



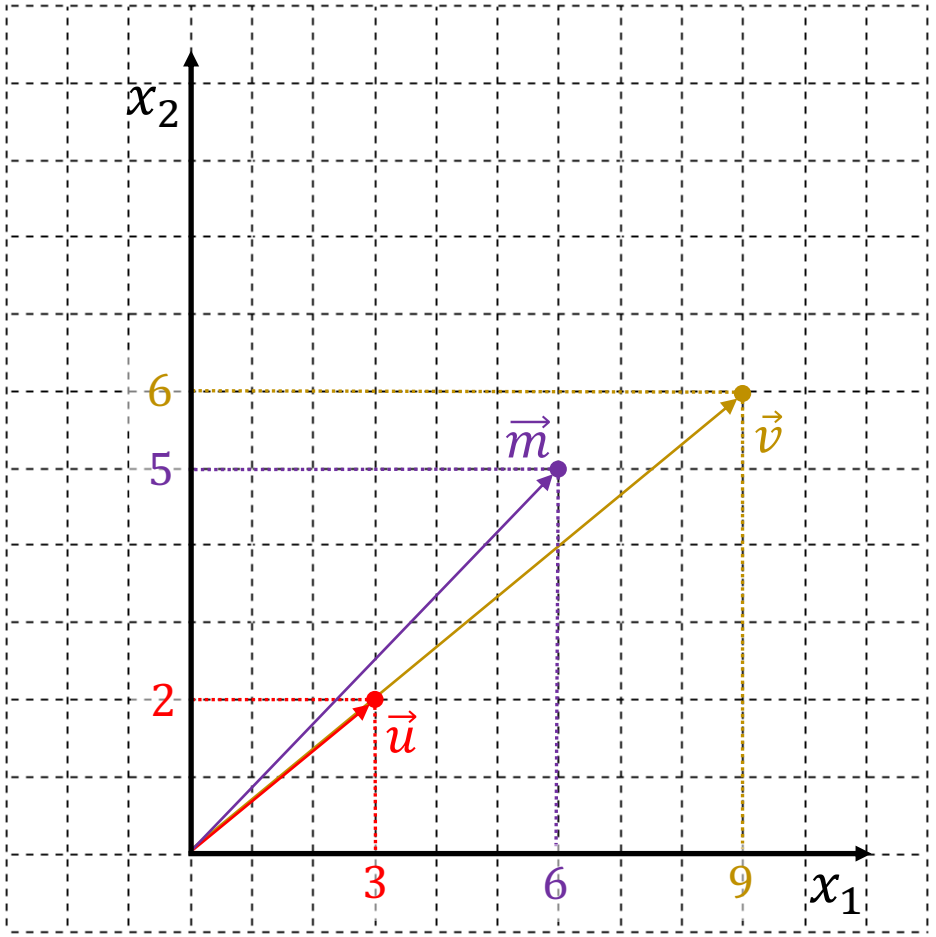
Tìm độ dài hình chiếu của $\vec{a}$ lên $\overrightarrow{v_1}$ và $\overrightarrow{v_2}$

Dot Product

$$\vec{u} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

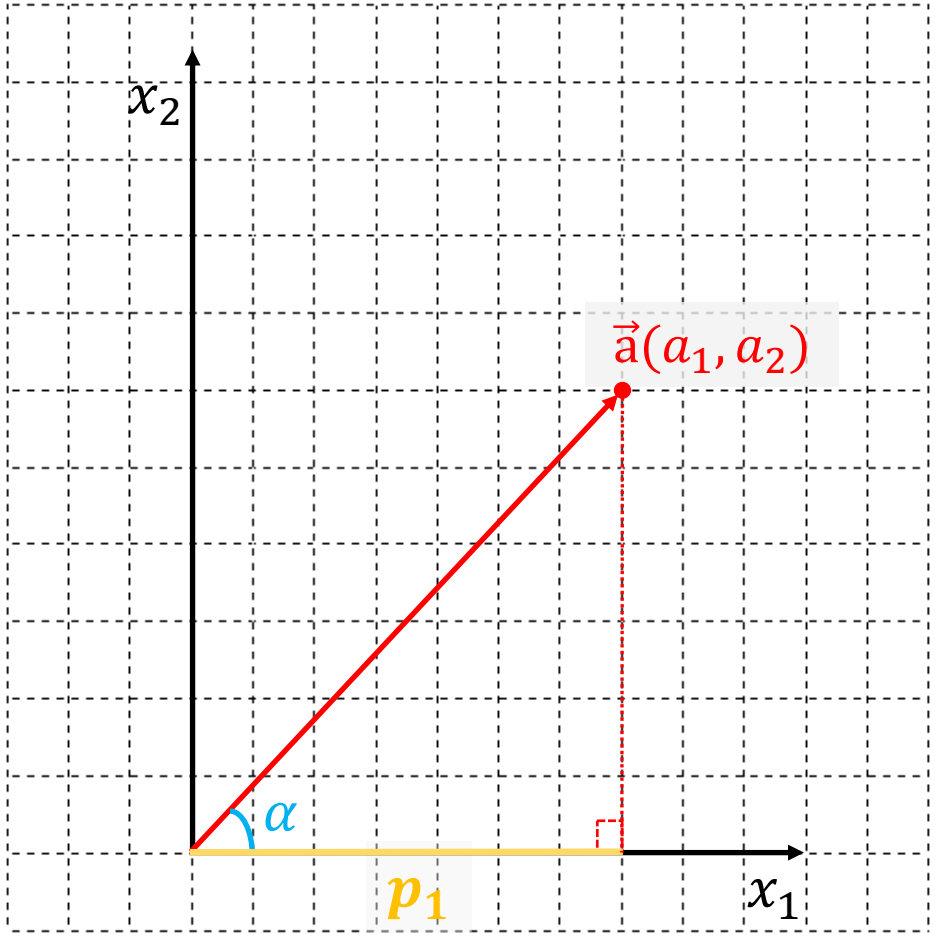


$$\vec{u} = \begin{pmatrix} 3 \\ 2 \end{pmatrix} \quad \vec{v} = 3 * \vec{u} = \begin{pmatrix} 9 \\ 6 \end{pmatrix} \quad \vec{m} = 3 + \vec{u} = \begin{pmatrix} 6 \\ 5 \end{pmatrix}$$

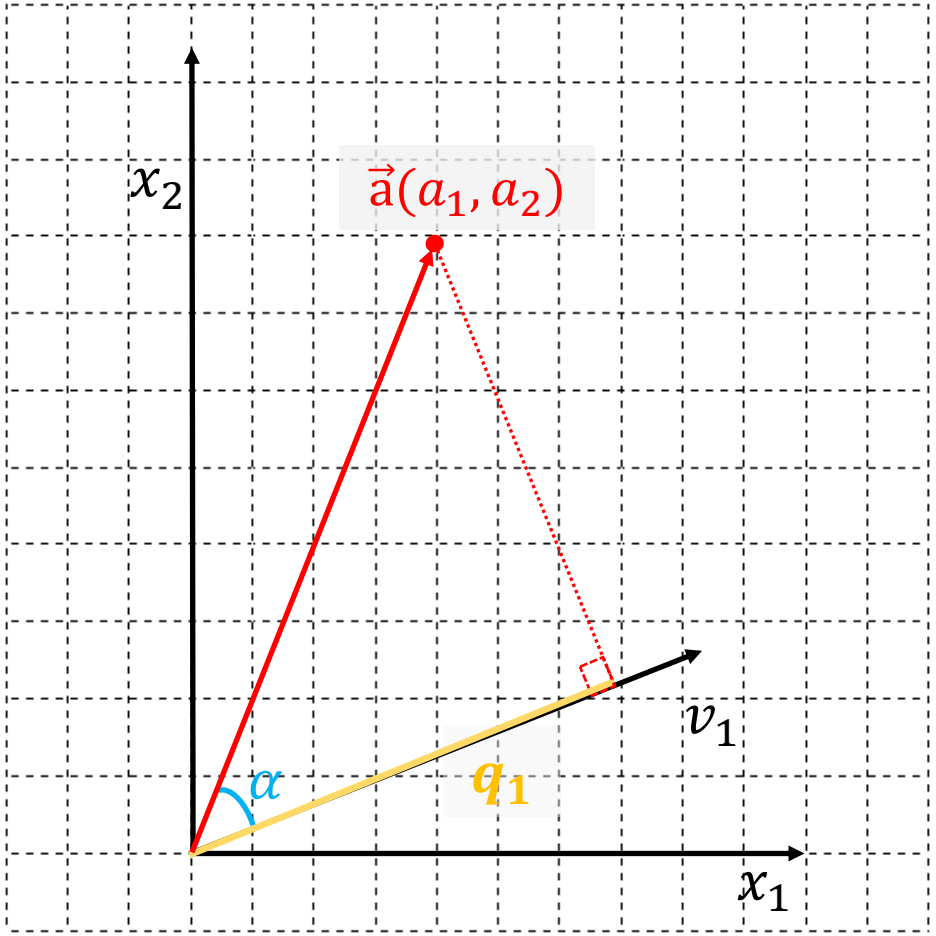


Dot Product

$\overrightarrow{x_1} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ $\overrightarrow{x_2} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

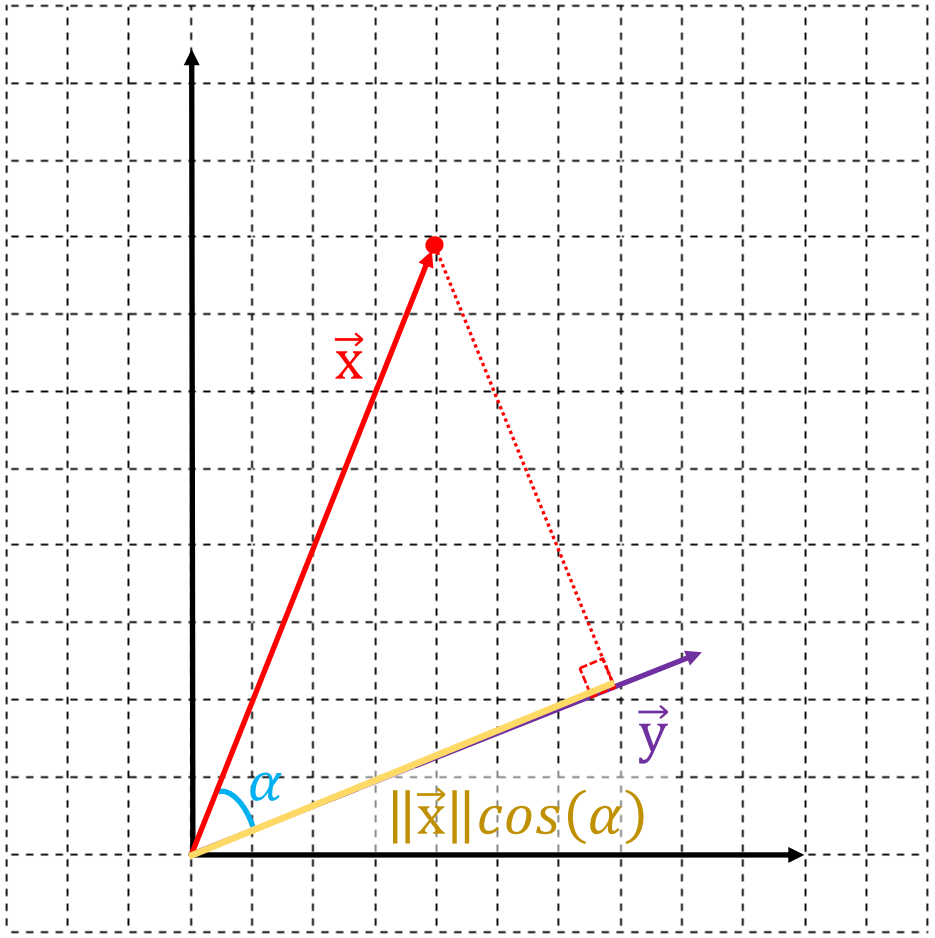


$\overrightarrow{v_1} = \begin{pmatrix} v_{1\_1} \\ v_{1\_2} \end{pmatrix}$ $\overrightarrow{v_2} = \begin{pmatrix} v_{2\_1} \\ v_{2\_2} \end{pmatrix}$



Example

Algebra	Geometry
$\vec{x}.\vec{y} = \sum_{1}^{n} x_i y_i$	$\vec{x}.\vec{y} = \|\vec{x}\|\|\vec{y}\| cos(\alpha)$




```python
import numpy as np

x = np.array([0, 5])
y = np.array([3, 3])
```

```python
# algebra
dproduct = np.dot(x, y)
print(dproduct)
```

```
15
```

```python
# geometry
x_length = np.linalg.norm(x)
y_length = np.linalg.norm(y)
cos_xy = np.cos(np.pi/4) # 45 degree

dproduct = x_length*y_length*cos_xy
print(dproduct)
```

```
14.999999999999998
```

Cosine similarity

❖ Cosine similarity (cs) is used to measure the similarity between two vectors

Let $\vec{x}$ and $\vec{y}$ be two vectors, cs is defined as

$$\text{cs}(\vec{x},\vec{y}) = \frac{\vec{x}.\vec{y}}{\|\vec{x}\|\|\vec{y}\|} = \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}}$$

Property 1: $\text{cs}(\vec{x},\vec{y}) = \text{cs}(a\vec{x}, b\vec{y})$; $ab > 0$

$$\text{cs}(a\vec{x}, b\vec{y}) = \frac{a\vec{x}.b\vec{y}}{\|a\vec{x}\|\|b\vec{y}\|} = \frac{\sum_1^n a x_i b y_i}{\sqrt{\sum_1^n a^2 x_i^2}\sqrt{\sum_1^n b^2 y_i^2}}$$

$$= \frac{ab\sum_1^n x_i y_i}{\sqrt{a^2\sum_1^n x_i^2}\sqrt{b^2\sum_1^n y_i^2}}$$

$$= \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}} = \text{cs}(\vec{x},\vec{y})$$

Property 2: $\text{cs}(\vec{x},\vec{y}) \neq \text{cs}(\vec{x}+c, \vec{y}+d)$

Example: $\vec{x} = [4,2,1,2]^T$

$\vec{y} = [1,2,2,0]^T$

$\vec{u} = 2\vec{x} = [8,4,2,4]^T$

$\vec{v} = 3\vec{y} = [3,6,6,0]^T$

$$\text{cs}(\vec{x},\vec{y}) = \frac{4*1+2*2+1*2+2*0}{\sqrt{4^2+2^2+1^2+2^2}\sqrt{1^2+2^2+2^2+0}}$$

$$= \frac{10}{\sqrt{25}\sqrt{9}} = \frac{10}{15} = 0.67$$

$$\text{cs}(\vec{u},\vec{v}) = \frac{8*3+4*6+2*6+4*0}{\sqrt{8^2+4^2+2^2+4^2}\sqrt{3^2+6^2+6^2+0}}$$

$$= \frac{60}{\sqrt{100}\sqrt{81}} = \frac{60}{90} = 0.67$$

$$= \text{cs}(\vec{x},\vec{y})$$

Cosine similarity

Cosine similarity (cs) is used to measure the similarity between two vectors

Let $\vec{x}$ and $\vec{y}$ be two vectors, cs is defined as

$$cs(\vec{x},\vec{y}) = \frac{\vec{x}.\vec{y}}{\|\vec{x}\|\|\vec{y}\|} = \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}}$$

Property 1: $cs(\vec{x},\vec{y}) = cs(a\vec{x}, b\vec{y})$; $ab > 0$

$$cs(a\vec{x}, b\vec{y}) = \frac{a\vec{x}.b\vec{y}}{\|a\vec{x}\|\|b\vec{y}\|} = \frac{\sum_1^n a x_i b y_i}{\sqrt{\sum_1^n a^2 x_i^2}\sqrt{\sum_1^n b^2 y_i^2}}$$

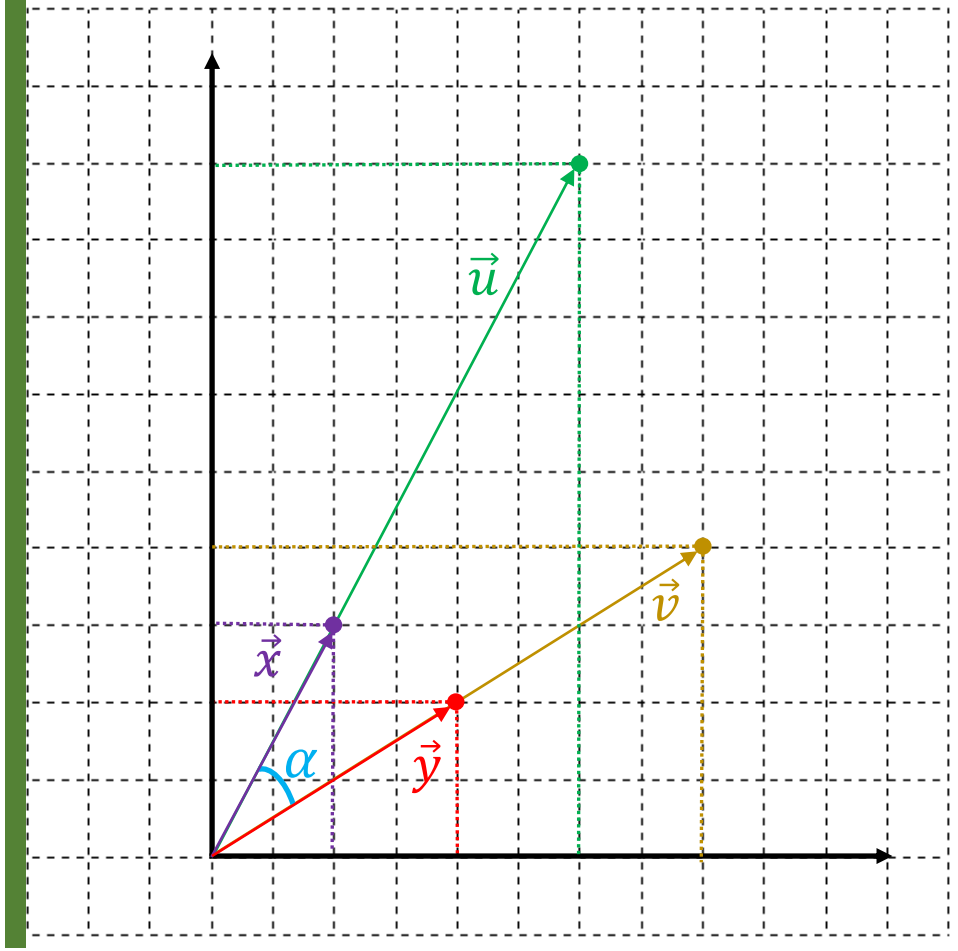
$$= \frac{ab\sum_1^n x_i y_i}{\sqrt{a^2\sum_1^n x_i^2}\sqrt{b^2\sum_1^n y_i^2}}$$

$$= \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}} = cs(\vec{x},\vec{y})$$

Property 2: $cs(\vec{x},\vec{y}) \neq cs(\vec{x}+c, \vec{y}+d)$

$\vec{x} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$ $\quad \vec{u} = 3 * \vec{x} = \begin{pmatrix} 6 \\ 9 \end{pmatrix}$

$\vec{y} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$ $\quad \vec{v} = 2 * \vec{y} = \begin{pmatrix} 8 \\ 4 \end{pmatrix}$



Cosine similarity

Cosine similarity (cs) is used to measure the similarity between two vectors

Let $\vec{x}$ and $\vec{y}$ be two vectors, cs is defined as

$$\text{cs}(\vec{x},\vec{y}) = \frac{\vec{x}.\vec{y}}{\|\vec{x}\|\|\vec{y}\|} = \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}}$$

Property 1: $\text{cs}(\vec{x},\vec{y}) = \text{cs}(a\vec{x}, b\vec{y})$; $ab > 0$

$$\text{cs}(a\vec{x}, b\vec{y}) = \frac{a\vec{x}.b\vec{y}}{\|a\vec{x}\|\|b\vec{y}\|} = \frac{\sum_1^n a x_i b y_i}{\sqrt{\sum_1^n a^2 x_i^2}\sqrt{\sum_1^n b^2 y_i^2}}$$

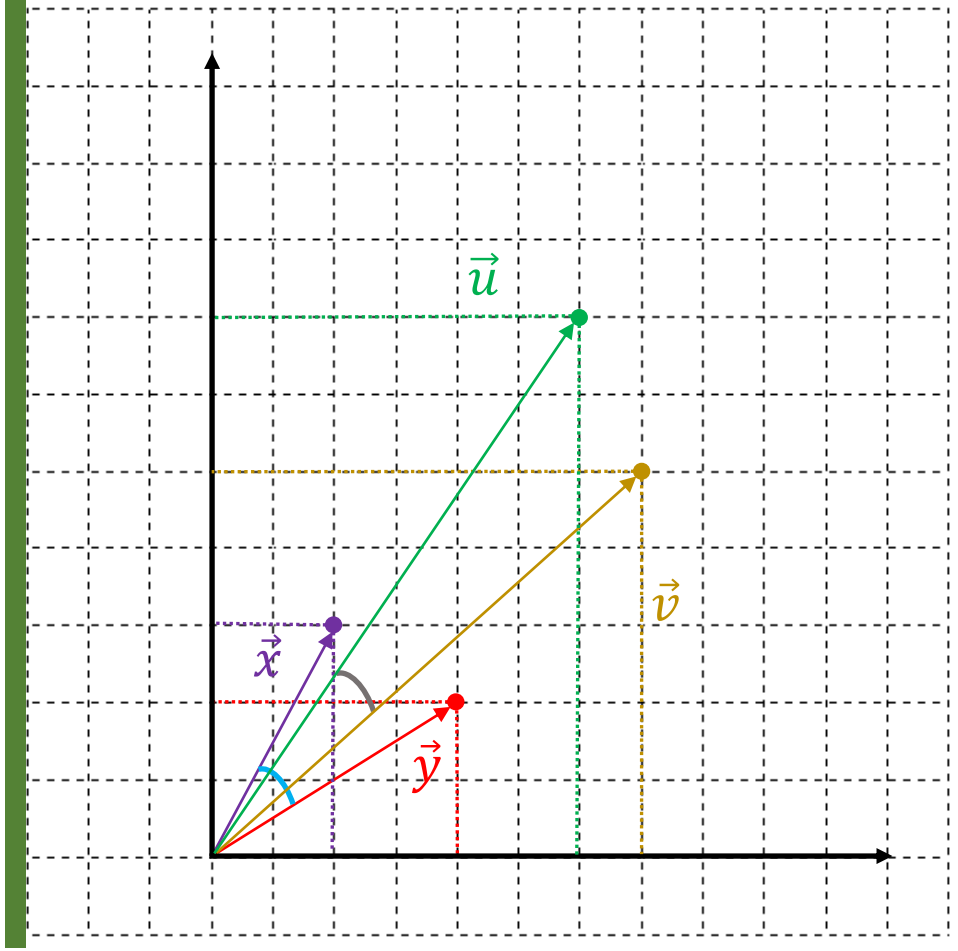
$$= \frac{ab\sum_1^n x_i y_i}{\sqrt{a^2\sum_1^n x_i^2}\sqrt{b^2\sum_1^n y_i^2}}$$

$$= \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}} = \text{cs}(\vec{x},\vec{y})$$

Property 2: $\text{cs}(\vec{x},\vec{y}) \neq \text{cs}(\vec{x}+c, \vec{y}+d)$

$\vec{x} = \begin{pmatrix}2\\3\end{pmatrix}$ $\quad$ $\vec{u} = \begin{pmatrix}4\\4\end{pmatrix} + \vec{x} = \begin{pmatrix}6\\7\end{pmatrix}$

$\vec{y} = \begin{pmatrix}4\\2\end{pmatrix}$ $\quad$ $\vec{v} = \begin{pmatrix}3\\3\end{pmatrix} + \vec{y} = \begin{pmatrix}7\\5\end{pmatrix}$



Cosine similarity

$$cs(\vec{x}, \vec{y}) = \frac{\vec{x}.\vec{y}}{\|\vec{x}\|\|\vec{y}\|} = \frac{\sum_1^n x_i y_i}{\sqrt{\sum_1^n x_i^2}\sqrt{\sum_1^n y_i^2}}$$

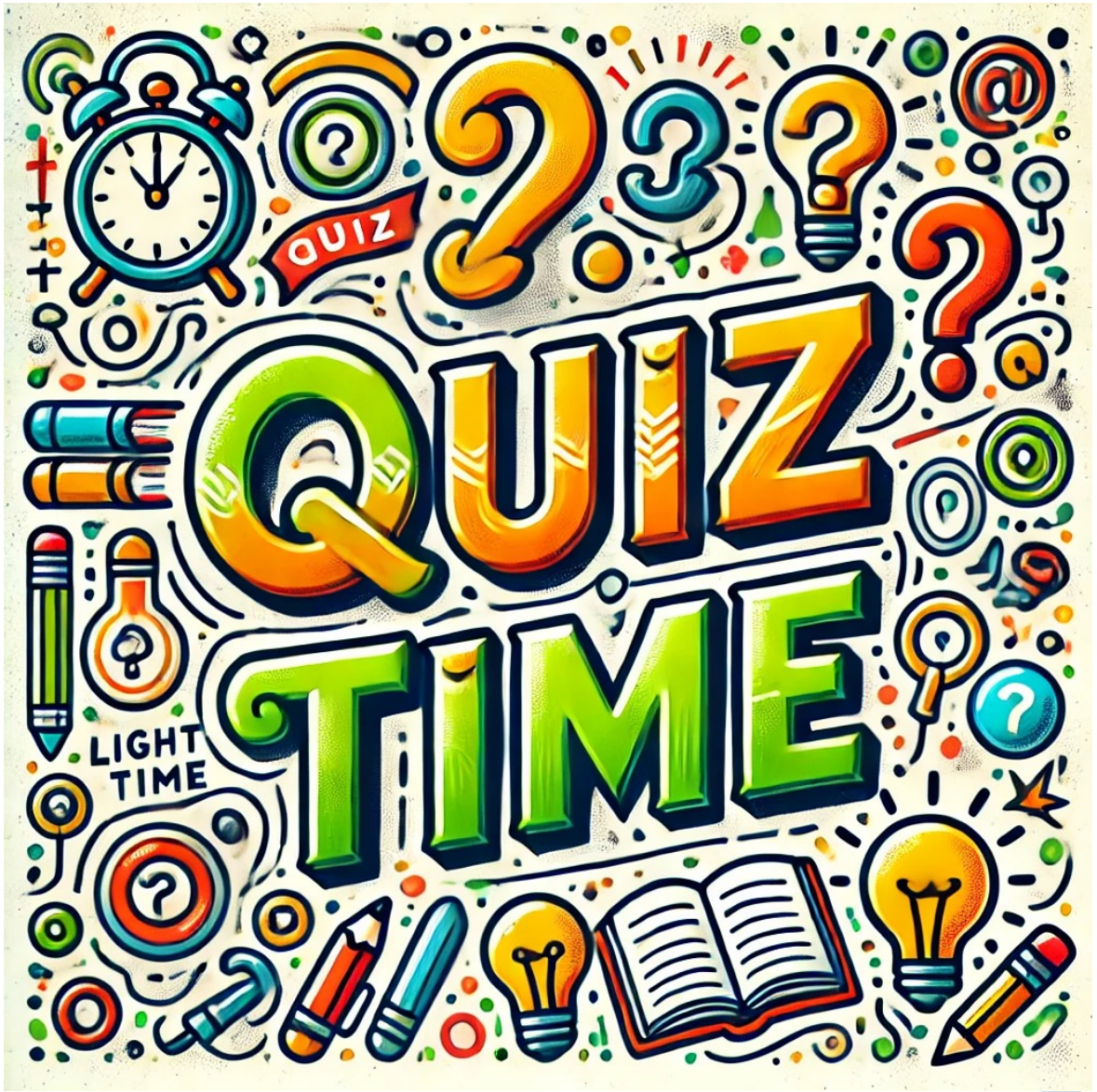
```python
import numpy as np

vector1 = np.array([5, 3, 2, 7])
vector2 = np.array([2, 9, 4, 1])

# compute dot product
dproduct = np.dot(vector1, vector2)

# compute norms
norm_1 = np.linalg.norm(vector1)
norm_2 = np.linalg.norm(vector2)

# compute similarity
cos_sim = dproduct / (norm_1*norm_2)
print(cos_sim)
```

```
0.552005787925351
```

❖ Implementation

```python
import math

def cosine_similarity(vector1, vector2):
    '''
    Compute dot product between two vectors
    Output is a floating-point number
    '''

    sumxy = sum([v1*v2 for v1, v2 in zip(vector1, vector2)])
    sumxx = sum([v1*v2 for v1, v2 in zip(vector1, vector1)])
    sumyy = sum([v1*v2 for v1, v2 in zip(vector2, vector2)])

    return sumxy/math.sqrt(sumxx*sumyy)

# test case
vector1 = [5, 3, 2, 7]
vector2 = [2, 9, 4, 1]

output = cosine_similarity(vector1,vector2)
print(output)
```

```
0.552005787925351
```


QUIZ
QUIZ
TIME
LIGHT
TIME

Outline

SECTION 1

Matrix Transformation

SECTION 2

Least-squares Estimation

SECTION 3

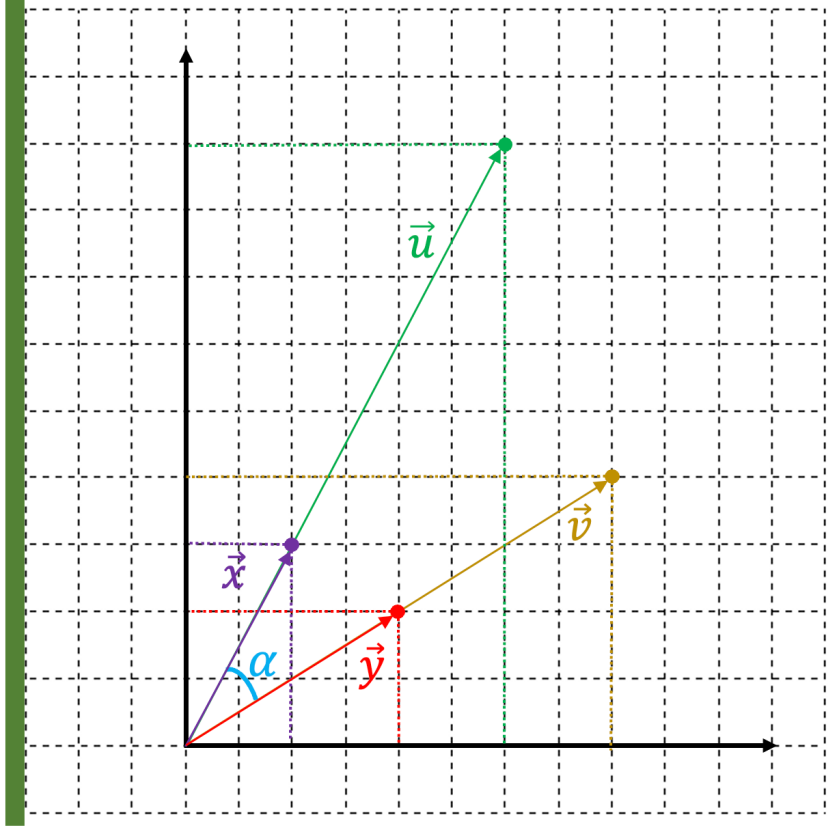
Cosine Similarity

SECTION 4

Applications

$\vec{x} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}$ $\vec{u} = 3 * \vec{x} = \begin{pmatrix} 6 \\ 9 \end{pmatrix}$

$\vec{y} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$ $\vec{v} = 2 * \vec{y} = \begin{pmatrix} 8 \\ 4 \end{pmatrix}$



Traffic Sign Matching

Traffic Sign Matching

❖ Using L1 and Cosine Similarity

Sign 1

Sign 2

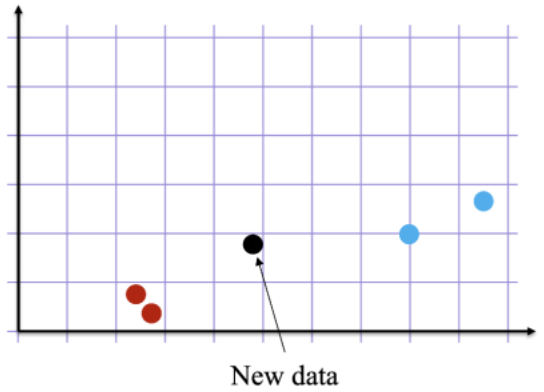
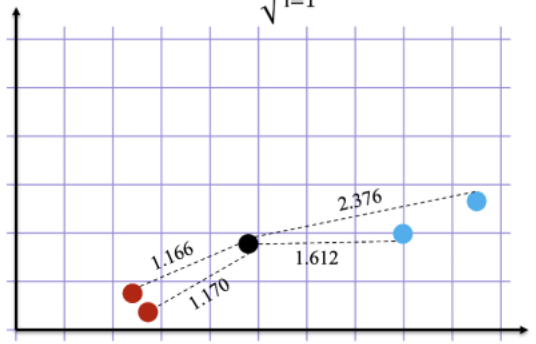
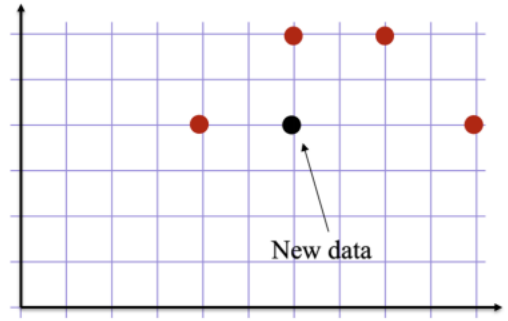
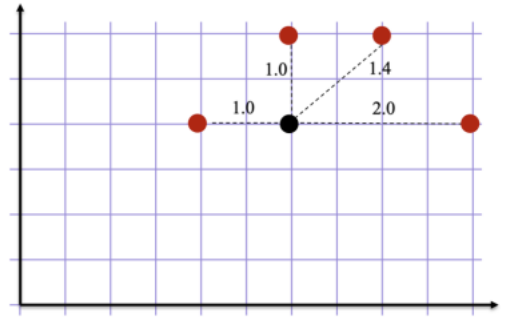
Sign 3

Sign 4

K-Neareast Neighbors

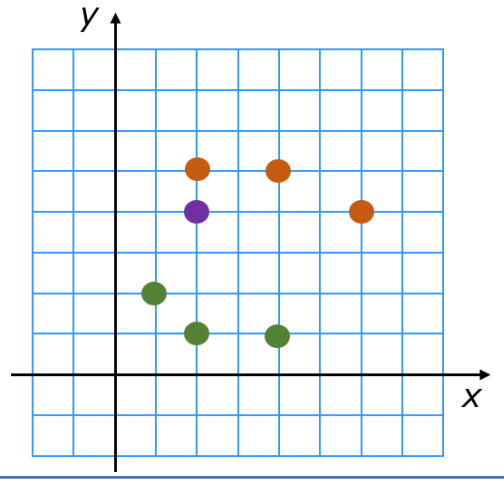
K-Nearest Neighbors

❖ Overview

	Step 1: Look at the data	Step 2: Calculate distances	Step 3: Find neighbours	Step 4: Vote on labels
Classification		Euclidean Distance $d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$ 	Ranking points 1 st 2 nd 3 rd 4 th Find the nearest neighbours by ranking points by increasing distance	K=3 Nearest neighbours 1 st 2 nd 3 rd # of votes 2 1 Vote on the predicted class labels based on the class of the k nearest neighbors
Regression			Ranking points 1 st 2 nd 3 rd 4 th Find the nearest neighbours by ranking points by increasing distance	K=4 Nearest neighbours 1 st 2 nd 3 rd 4 th $Y_{pred} = \frac{1}{k}\sum_{x \in NB} y_x$ Compute the mean value of the k nearest neighbors

KNN Algorithm

Petal_Length (cm)	Petal_Width (cm)	Label
1.4	0.2	0
1.3	0.4	0
1.4	0.3	0
4	1	1
4.7	1.4	1
3.6	1.3	1



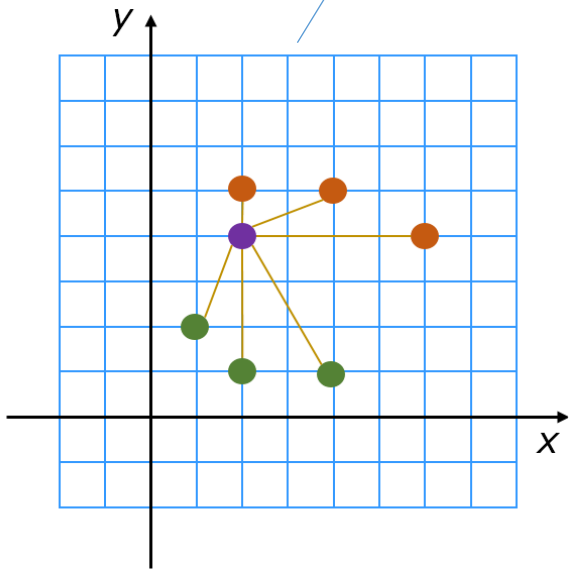
Prepare data and select K

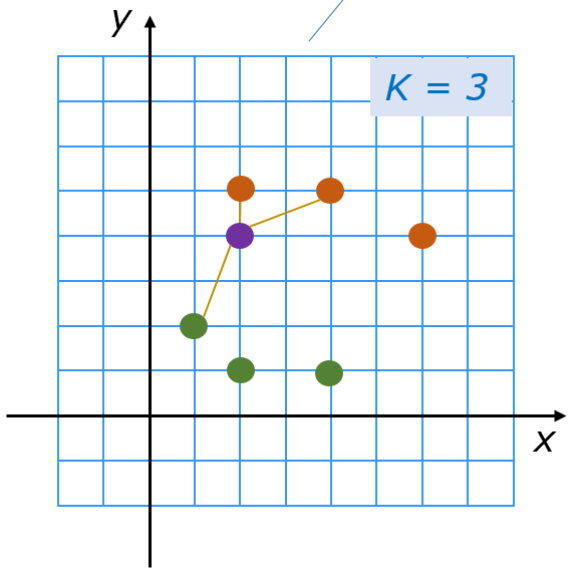
Compute distances between a testing point and points in training data

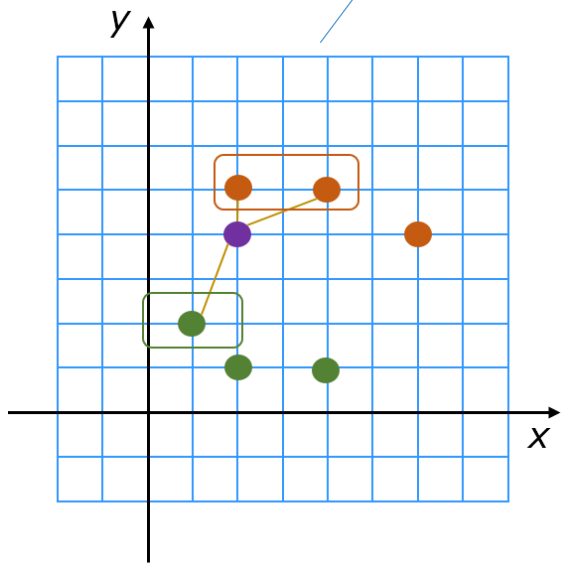
Take the K nearest neighbors

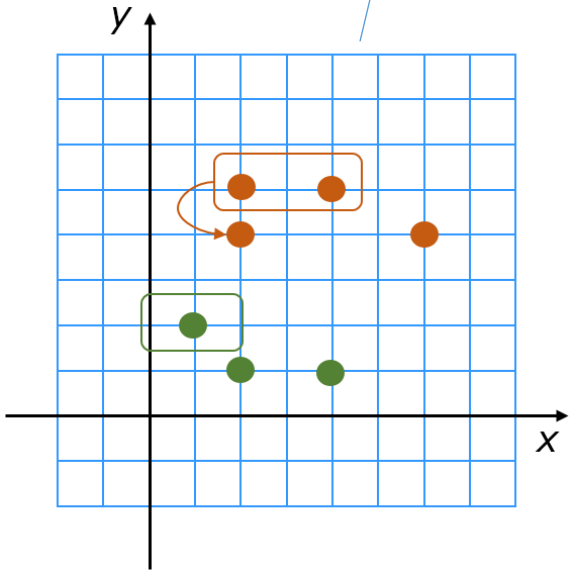
Voting

Output









K-Nearest Neighbors

❖ Example

Petal_Length	Label	Distance
1.4	0	1
1	0	1.4
1.5	0	0.9
3.1	1	0.7
3.7	1	1.3
4.1	1	1.7

New input data
x_test = 2.4

Petal_Length	Label	Distance
1.4	0	1
1	0	1.4
1.5	0	0.9
3.1	1	0.7
3.7	1	1.3
4.1	1	1.7

k=1
→ y_test = 1

k=3
→ y_test = 0

AI VIETNAM

K-Nearest Neighbors

❖ Example

Petal_Length	Petal_Width	Label	Distance
1.4	0.2	0	1.166
1.3	0.4	0	1.17
1.4	0.3	0	1.118
4	1	1	1.612
4.7	1.4	1	2.376
3.6	1.3	1	1.3

New input data
x_test = (2.4, 0.8)

K = 1

K = 3

AI VIETNAM

blank page